



CuTe -- CUDA Tensors

Cris Cecka

Principal Research Scientist, NVIDIA

GPU MODE -- Sept 4th, 2026



PyCuTe

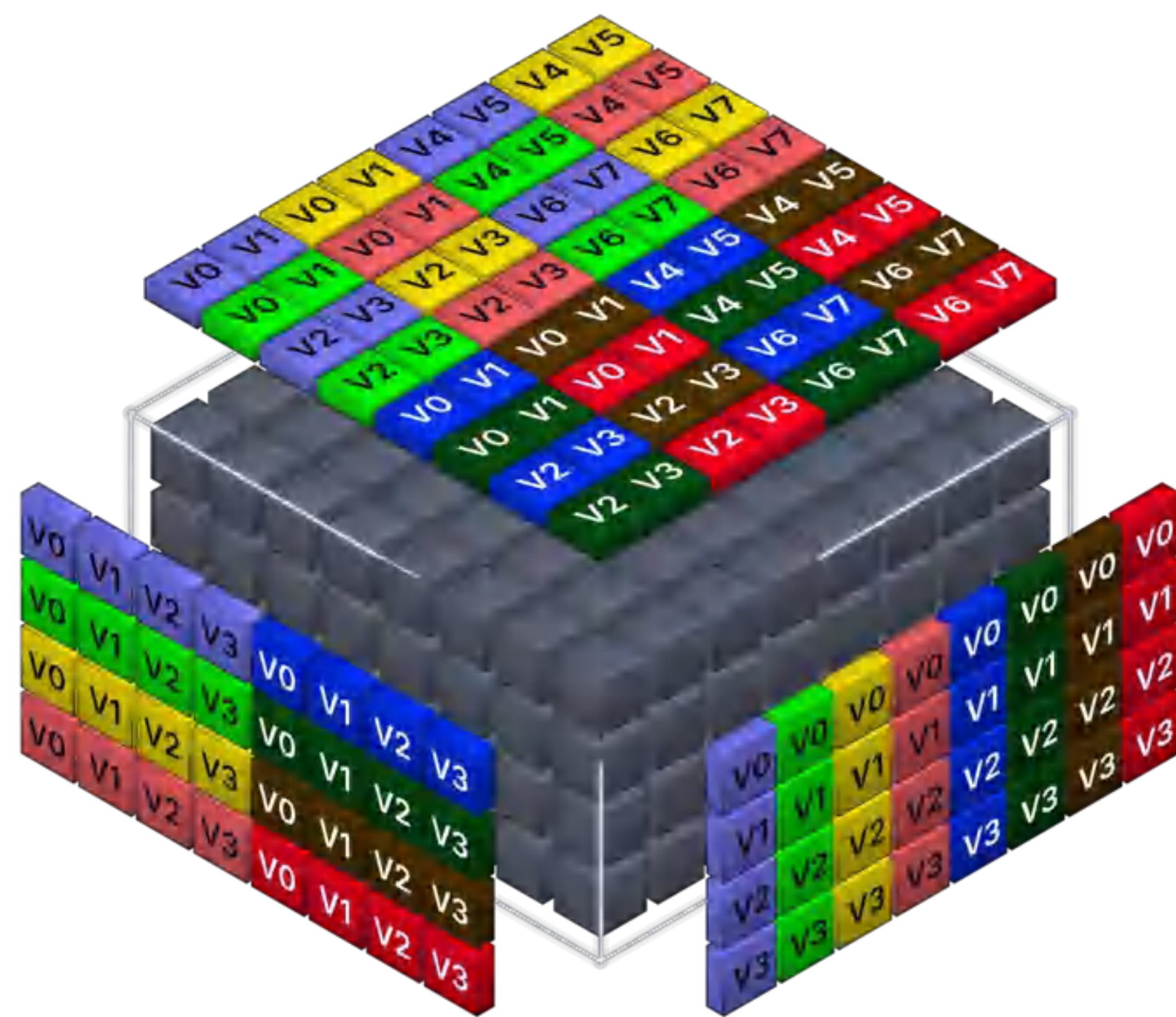
- Integrations

- Better tensor abstractions

- Better tensor algorithms

- Abstracted Strides

- All about Swizzles



PyCuTe – reference implementation

<https://github.com/NVlabs/CuTe>

Ongoing work:

- Reference examples for *layout analysis* in
 - COPY
 - GEMM
 - SMEM Layout solver
 - Tile-level metaprogramming
- Integrations
 - CuTeDSL
 - CuTeIR
 - cuda.lang (enable meta-programming)

CuTe LAYOUT REPRESENTATION AND ALGEBRA

A PREPRINT

Cris Cecka
NVIDIA Research
ccecka@nvidia.com

ABSTRACT

Modern architectures for high-performance computing and deep learning increasingly incorporate specialized tensor instructions, including tensor cores for matrix multiplication and hardware-optimized copy operations for multi-dimensional data. These instructions prescribe fixed, often complex data layouts that must be correctly propagated through the entire execution pipeline to ensure both correctness and optimal performance. We present CuTe, a novel mathematical specification for representing and manipulating tensors. CuTe introduces two key innovations: (1) a hierarchical layout representation that directly extends traditional flat-shape and flat-stride tensor representations, enabling the representation of complex mappings required by modern hardware instructions, and (2) a rich algebra of layout operations – including concatenation, coalescence, composition, complementation, division, tiling, and inversion – that enables sophisticated layout manipulation, derivation, verification, and static analysis. CuTe layouts provide a framework for managing both data layouts and thread arrangements in GPU kernels, while the layout algebra enables powerful compile-time reasoning about layout properties and the expression of generic tensor transformations.

In this work, we demonstrate that CuTe’s abstractions significantly aid software development compared to traditional approaches, promote compile-time verification of architecturally prescribed layouts, facilitate the implementation of algorithmic primitives that generalize to a wide range of applications, and enable the concise expression of tiling and partitioning patterns required by modern specialized tensor instructions.

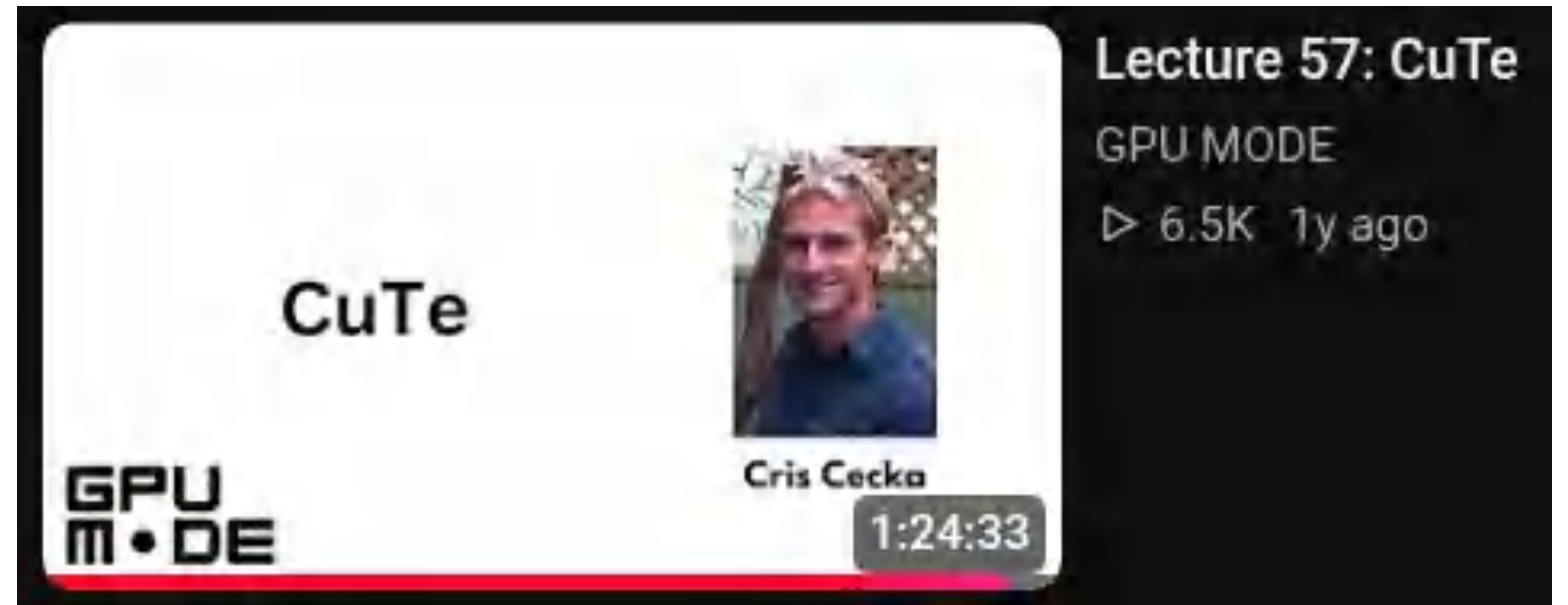
CuTe has been successfully deployed in production systems, forming the foundation of NVIDIA’s CUTLASS library and a number of related efforts including CuTe DSL. We provide both mathematical foundations and a reference implementation in Python (PyCuTe) [1], establishing CuTe as a principled approach to layout-aware programming for the tensor-centric computing era.

<https://arxiv.org/pdf/2603.02298>

Previous Talks

References

- [GPU MODE Lecture 57: CuTe](#) (Cris Cecka)
- GPU MODE Lecture 114: PyCuTe (Cris Cecka)



- [GPU MODE Lecture 86: CuTeDSL](#) (Alex Zhang, Vicki Wang)
- [GTC 2025: Blackwell with CuTe and CUTLASS](#) (Cris Cecka, Mihir Awatramani)
- [GTC 2025: Tensor Cores with CUTLASS 4.0](#) (Brandon Sun, Khiero Bando)
- [LLVM 2025: CUTLASS/CuTe DSL](#) (Guray Ozen)
- [Open Source 2023: CUTLASS C++](#) (Vijay Thakkar, Aniket Shivam)

Representation and Algebra

Why they matter

- Can you **express** the index mapping your hardware requires?
 - Can you **manipulate** the index mappings across levels?
 - Can you **evaluate** the index mapping efficiently (at run-time)?
 - Can you **reason** about the index mapping?
-
- Index mappings
 - Memory layouts (L): $\text{Tensor coordinate} \rightarrow \text{memory offset}$
 - Tilings (T): $(m_tile, n_tile, m_grid, n_grid) \leftrightarrow \text{Tensor coordinate } (m, n)$
 - Access distributions (TV): $(agent_id, elem_id) \leftrightarrow \text{Tensor coordinate}$
 - Combinations
 - Memory access patterns: $L \circ TV : (agent_id, elem_id) \rightarrow \text{memory offset}$
 - Tile access patterns: $T^{-1} \circ TV : (agent_id, elem_id) \rightarrow (m_tile, n_tile, m_grid, n_grid)$
 - Memory transfers: $L0 \circ L1^{-1} : \text{memory1 offset} \rightarrow \text{memory0 offset}$
 - Access transfers: $L0^{-1} \circ L1 : (agent1_id, elem1_id) \rightarrow (agent0_id, elem0_id)$

Mathematical properties

Stay CuTe

- **Scope:** Can represent any computable index mapping
 - CuTe (int) >> Flat shapes:strides
 - CuTe (F2) >> CuTe (int)
 - CuTe (Composed)
- **Efficiency and interpretability:** Simple mappings have simple representations
 - CuTe: Immediate generalization from existing `shape:stride` representations.
- **Closure:** Operations on mappings produce mappings of the same representation
 - CuTe: Compatibility and admissibility
- **Tension**
 - Closure often restricts scope
 - Scope restricts closure
 - Interpretability restricts scope

Software vs Compilers in the Agentic Age

Metaprogramming vs Compilation

How do Programs, Metaprograms, Compilers, and AI Agents evolve together?

- Agents?
 - Perform optimization and low-level programming
 - With a compiler "harness" that provides deterministic correctness checks to make the search space well-defined
 - With metaprogrammed abstractions and algorithms as recipes/examples and starting points for optimization search
- Metaprograms?
 - **Localized** and transparent static algorithms for dispatch, verification, and correctness
 - Ability to provide **transparent** heuristics/abstractions
- Compilers?
 - Avoid hand-crafted optimization heuristics
 - Instead, focus on **global** analysis, verification, and tooling that help agents (and humans) optimize safely and effectively
 - "Compiler knobs and hints are NOT a Programming Model"

**Agents increasingly own optimization and code generation,
Compilers evolve into agent-facing platforms that provide correctness verification, analysis, and feedback loops to guide agentic optimization.
Metaprograms provide transparent structured representations, static validation, local code transformations, and case examples.**



Integrations

CuTeDSL Integration

Is Done*

- PyCuTe works out of the box with CuTeDSL, no modifications required.

```
# -- Adapter -----  
# Dynamic extents are PyCuTe integers; DSL pointers are PyCuTe accessors.  
import cutlass.cute as cute  
from cutlass.base.dsl.typing import Integer as _CuTeInteger  
import pycute  
pycute.register_integer_type(_CuTeInteger)  
pycute.MutableAccessor.register(cute.Pointer)  
  
def from_dsl(t: cute.Tensor) -> pycute.Tensor:  
    """  
    View a `cute.Tensor` as a `pycute.Tensor`.  
    """  
    return pycute.Tensor(t.iterator, pycute.Layout(t.layout.shape, t.layout.stride))  
  
def to_dsl(T: pycute.Tensor) -> cute.Tensor:  
    """  
    View a `pycute.Tensor` as a `cute.Tensor`.  
    """  
    return cute.make_tensor(T.accessor, cute.make_layout(pycute.shape(T.layout),  
                                                         stride=pycute.stride(T.layout)))
```

- Nothing is copied and no IR is emitted – this is compile-time metaprogramming.

pycute.Tensor

vs cute.Tensor

```
@cute.kernel
def copy_kernel(a: cute.Tensor, b: cute.Tensor):
    tid, _, _ = cute.arch.thread_idx()

    A = from_dsl(a)
    B = from_dsl(b)

    gA = zipped divide(A, (4,3))[:, tid]
    gB = zipped divide(B, (4,3))[:, tid]

    pycute.alg.copy(gA, gB)
```

- CuTeDSL is a tracing compiler, so all of Python is available.

PyCuTe for static and dynamic

CuTeDSL + PyCuTe

```
@cute.jit
def layout_size_static(M: cutlass.Constexpr,
                      N: cutlass.Constexpr):
    l = pycute.Layout((M, N), (N, 1))
    cute.printf("static size = {}",
               pycute.size(l))

@cute.jit
def layout_divide(M: cutlass.Int32,
                 N: cutlass.Int32):
    res = pycute.zipped_divide(
        pycute.Layout((M, N), (N, 1)), (2, 2))
```

- `Constexpr` folds to a constant
- `Int32` stays symbolic and lowers to a multiply
- The `(2, 2)` tiler is interpreted as static

cuda.lang Integration

- cuda.lang is an AST compiler, and does require changes to PyCuTe to interop
- The boundary between trace-time code and compiled code must be declared

<code>@static_def</code>	mark a function trace-time
<code>@dataclass(frozen=True)</code>	make layouts and tensors value types

```
@dataclass(frozen=True, init=False)
class Layout(LayoutBase):
    shape: Shape
    stride: Stride
```

```
# pycute/algebra.py
@ModeOpDecorator
+@_cuda_lang_static_def
def zipped_divide(A, B, *, mode=()):
```

```
# pycute/layout.py
+@_cuda_lang_static_def
def __call__(self, *crd) -> StrideScalar:
    crd = crd[0] if len(crd) == 1 else crd
    return inner_product(
        idx2crd(crd, self.shape), self.stride)
```


Better Tensor Abstractions

CuTe (CUDA Tensor) Framework

Tensors *a.k.a.* Arrays

Common representation has 3 basic components

1. **Shape** defining valid n-D coordinates

a	b	c
d	e	f

Shape = (2, 3)

Rank = 2

2. **Layout** function mapping n-D coordinates to integers

$$\text{layout} : \mathbb{Z}^n \rightarrow \mathbb{Z}$$

3. **Accessor** mapping integers to values (or references)

e.g., a pointer in C, an iterator in C++, an array in Python, *etc.*

Layout functions

Linear combination of coordinates

Common practice defines layout by:

`index = inner_product(coordinate, stride)`

with one stride value per coordinate dimension

a	b	c
d	e	f

shape: (2, 3)



stride: (3, 1)

a	b	c	d	e	f
---	---	---	---	---	---

$idx = i*3 + j*1$

Examples in Python

Most roughly follow the pattern established by NumPy

```
>>> import torch
```

```
>>> x = torch.tensor([[1,2,3], [4,5,6]])  
tensor([[1, 2, 3],  
        [4, 5, 6]])
```

```
>>> x.shape  
torch.Size([2, 3])
```

```
>>> x.stride()  
(3, 1)
```

```
>>> x.ravel()  
tensor([1, 2, 3, 4, 5, 6])
```

```
>>> import numpy
```

```
>>> y = numpy.array([[1,2,3], [4,5,6]])  
array([[1, 2, 3],  
       [4, 5, 6]])
```

```
>>> y.shape  
(2, 3)
```

```
>>> y.strides      # 8x for byte strides  
(24, 8)
```

```
>>> y.ravel()  
array([1, 2, 3, 4, 5, 6])
```


C++ array view `std::mdspan`

Abstraction for multi-dimensional data introduced in C++23

```
template<
    class T,
    class Extents,
    class LayoutPolicy = std::layout_right,
    class AccessorPolicy = std::default_accessor<T>
> class mdspan;
```


std::mdspan

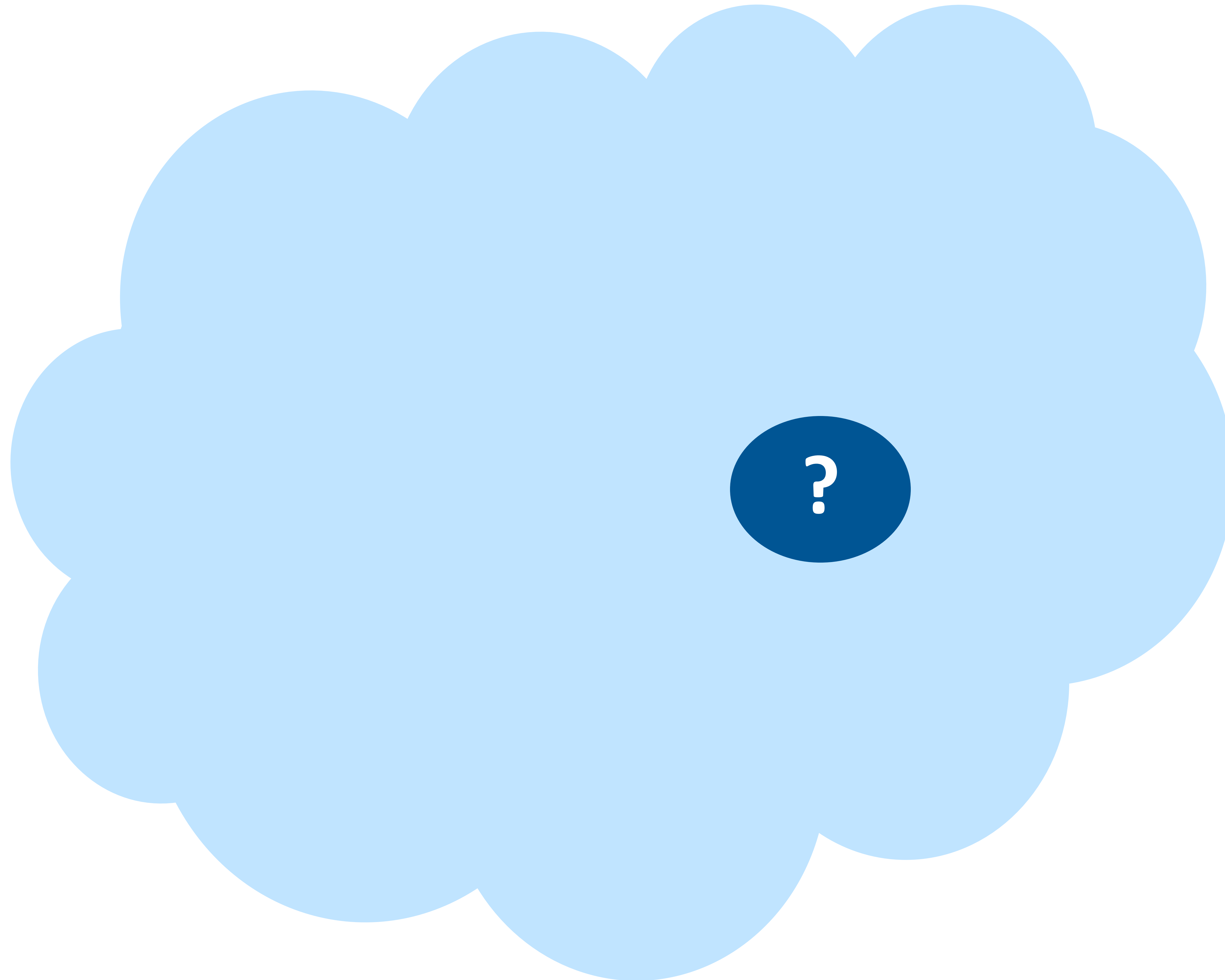
Abstraction for multi-dimensional data introduced in C++23

```
// container for actual values
vector<float> storage(20);

// create a view of this data as a 4x5 array
mdspan x{storage.data(), 4, 5};

// access multi-dimensional data
for(size_t c = 0; c < x.extent(1); ++c) {
    for(size_t r = 0; r < x.extent(0); ++r) {
        x[r, c] = (r + 1.0f) + (c + 1.0f) * x.extent(0);
    }
}
```


What layouts can we represent?



The most common layouts

Functions from coordinate to index

Logical

a	b	c
d	e	f

Shape: (2, 3)
Stride: (1, 2)

Column-major

a	b	c
d	e	f

Shape: (2, 3)
Stride: (3, 1)

Row-major

a	b	c
d	e	f

Shape: (2, 3)
Stride: (1, 4)

Padded Col-major

		(_, _, 1)
		e f
a b		
c d		
		h
		(_, _, 0)

Shape: (2, 2, 2)
Stride: (4, 1, 2)

Tensor layout

Physical

a	d	b	e	c	f
---	---	---	---	---	---

$$\text{idx} = i*1 + j*2$$

a	b	c	d	e	f
---	---	---	---	---	---

$$\text{idx} = i*3 + j*1$$

a	d			b	e			c	f		
---	---	--	--	---	---	--	--	---	---	--	--

$$\text{idx} = i*1 + j*4$$

a	b	e	f	c	d	g	h
---	---	---	---	---	---	---	---

$$\text{idx} = \text{inner_product}(\text{crd}, \text{stride})$$

Generalized column/row-major layouts

Both can be calculated from the shape of any array

Generalized column-major:

```
strides = prefix_product(shape)
```

Generalized row-major:

```
strides = reverse(prefix_product(reverse(shape)))
```

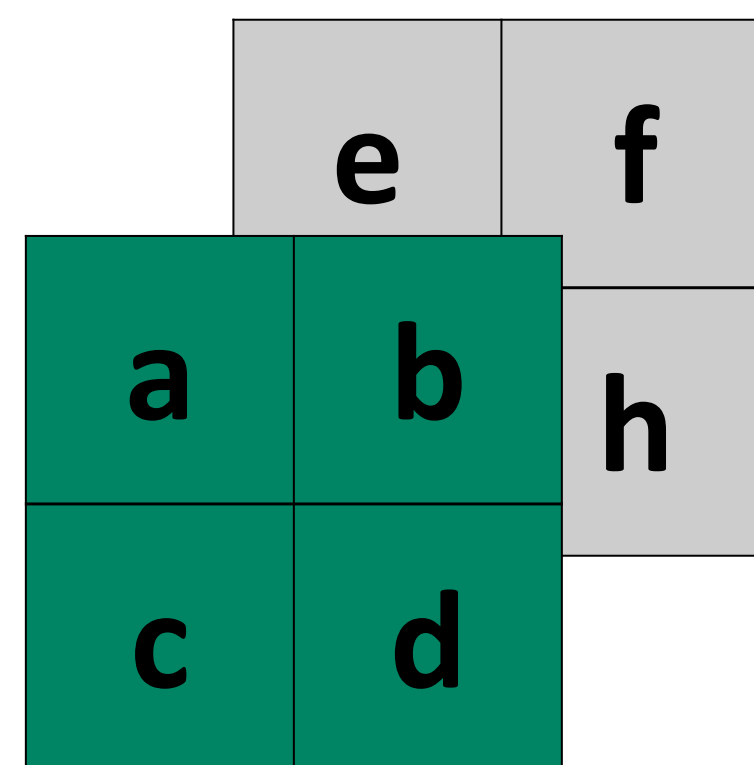

Other permutations certainly work

Which starts to matter for 3D arrays and beyond

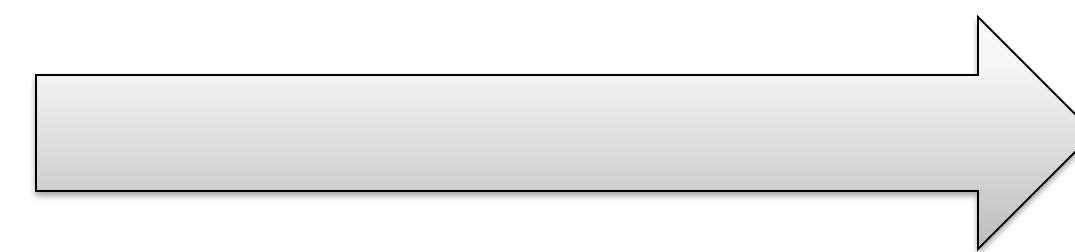
```
strides = unpermute(prefix_product(permute(shape)))
```

for any other choice of permutation.

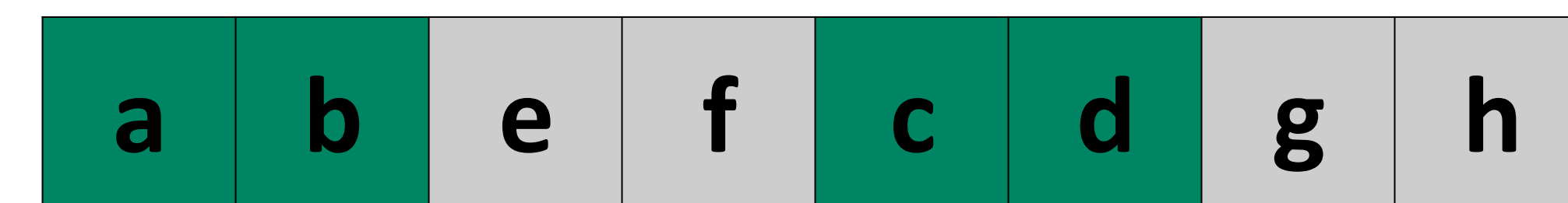
Logical Tensor



Shape: (2, 2, 2)
Stride: (4, 1, 2)



Physical Storage

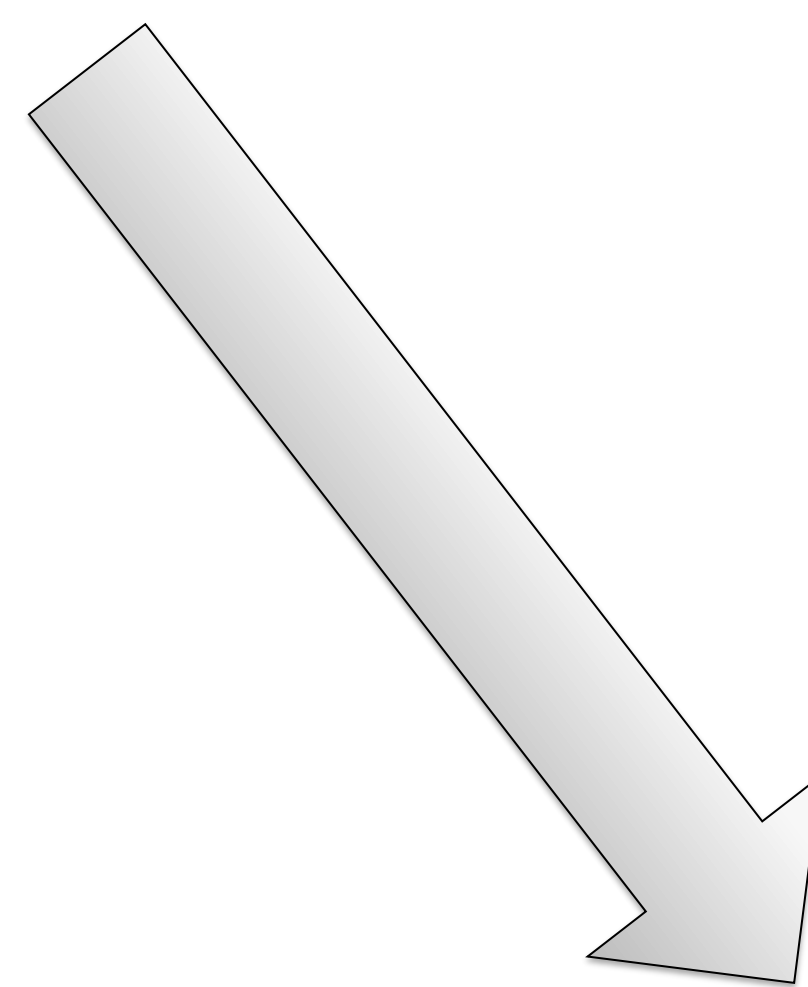


The limits of stride representation

Suppose we want to fold a tensor in different ways while maintaining storage order

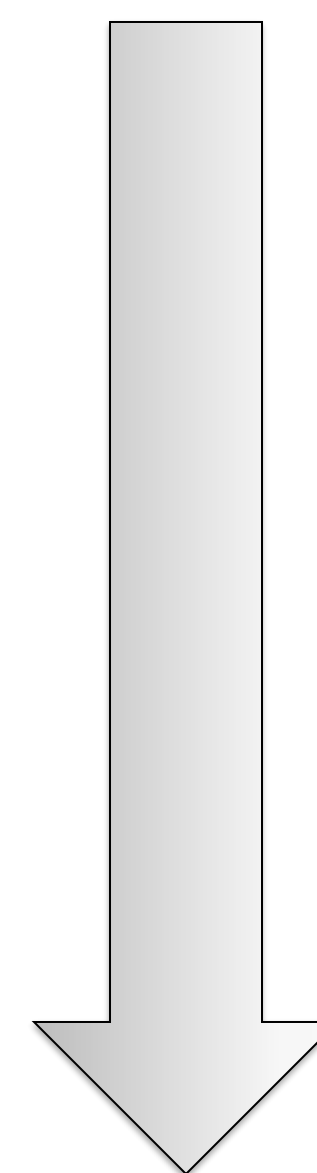
	e	f
a	b	h
c	d	

Shape: (2, 2, 2)
Stride: (4, 1, 2)



a	b	e	f
c	d	g	h

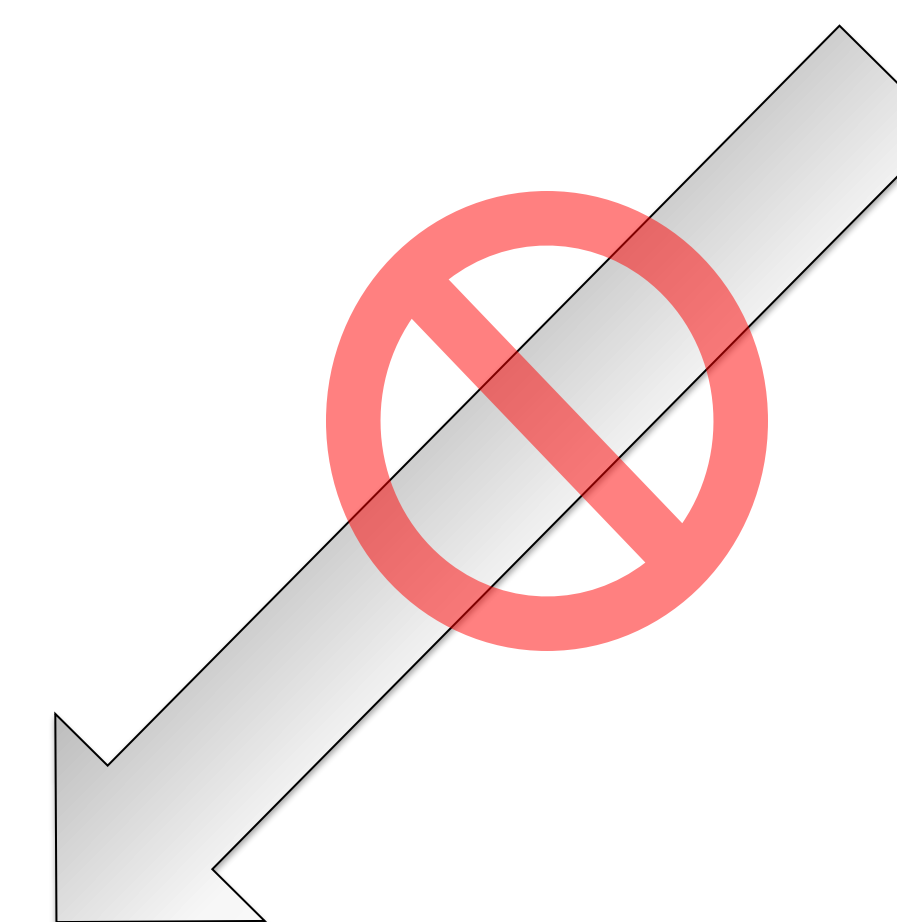
Shape: (2, 4)
Stride: (4, 1)



a	b	e	f	c	d	g	h
---	---	---	---	---	---	---	---

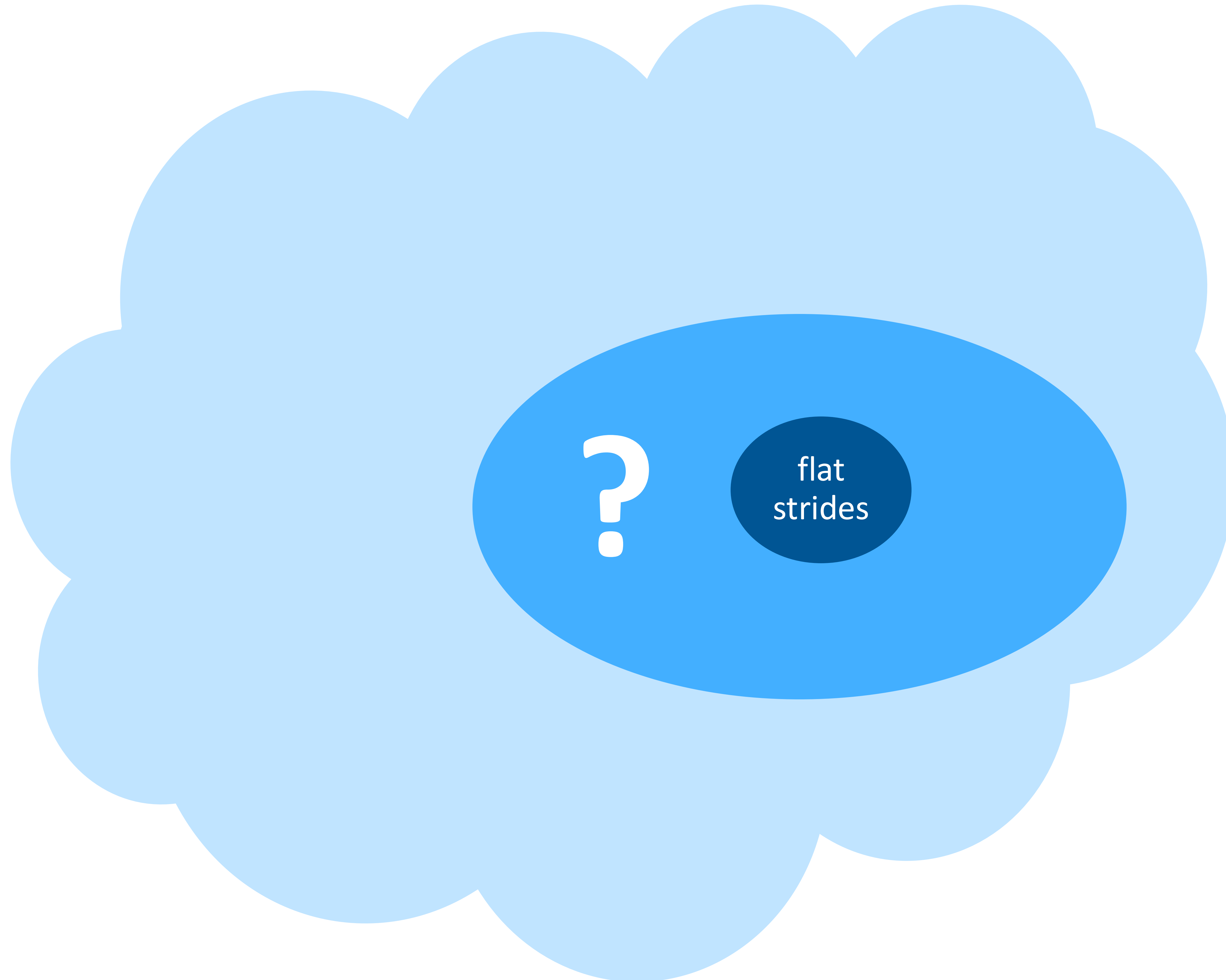
a	b
c	d
e	f
g	h

Shape: (4, 2)
Stride: (?, 1)



Can we expand the representable set?

Traditional strides don't cover everything we want



Key Idea #1: Generalize to hierarchical shapes / strides

Significantly extends the set of layout functions we can represent

a	b
c	d
e	f
g	h



a	b	e	f	c	d	g	h
---	---	---	---	---	---	---	---

`idx = inner_product(crd, stride)`

~~Shape: (4, 2)~~

~~Stride: (?, 1)~~

Shape: ((2, 2), 2)

Stride: ((4, 2), 1)

Hierarchical Layout

Example: More multi-modes

Shape: $((2, (2, 2)), (2, (2, 2)))$

Stride: $((1, (4, 16)), (2, (8, 32)))$

i \ j	0	1	2	3	4	5	6	7
0	0	2	8	10	32	34	40	42
1	1	3	9	11	33	35	41	43
2	4	6	12	14	36	38	44	46
3	5	7	13	15	37	39	45	47
4	16	18	24	26	48	50	56	58
5	17	19	25	27	49	51	57	59
6	20	22	28	30	52	54	60	62
7	21	23	29	31	53	55	61	63

- Logical coordinates are 1D, 2D, hD

$$A(37) = 49$$

$$A(5, 4) = 49$$

$$A((1, 2), (0, 2)) = 49$$

$$A((1, (0, 1)), (0, (0, 1))) = 49$$

- Slice along logical sub-boundaries

$$A(_, 2) = [8; 9; 12; 13; 24; 25; 28; 29]$$

$$A((_, 1), (_, 2)) = [36, 38; 37, 39]$$

From hand-written functions to declarative layouts

Dramatic reduction in code complexity

```
/// Returns the offset of a coordinate in linear memory.
CUTLASS_HOST_DEVICE
LongIndex operator()(TensorCoord const &coord) const {
    int vec_contiguous_idx = coord.contiguous() / kElementsPerAccess;
    int vec_strided_idx = coord.strided() / kFactor;

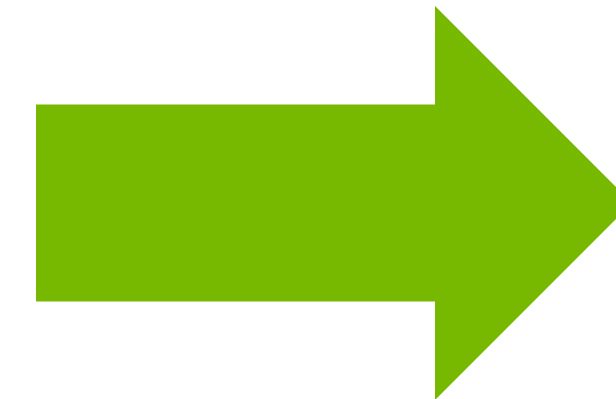
    int tile_contiguous_idx =
        vec_contiguous_idx / (TileShape::kContiguous / kFactor);
    int tile_contiguous_residual =
        vec_contiguous_idx % (TileShape::kContiguous / kFactor) +
        ((coord.strided() % kFactor) * (TileShape::kContiguous / kFactor));
    int tile_strided_residual = vec_strided_idx % TileShape::kStrided;

    int partition_contiguous_idx =
        tile_contiguous_residual / PartitionShape::kContiguous;
    int partition_strided_idx =
        tile_strided_residual / PartitionShape::kStrided;
    int partition_contiguous_residual =
        tile_contiguous_residual % PartitionShape::kContiguous;
    int partition_strided_residual =
        tile_strided_residual % PartitionShape::kStrided;

    int permuted_vec_contiguous_within_partition =
        partition_contiguous_residual ^ (partition_strided_residual % 4);
    int permuted_partition_contiguous_within_tile =
        partition_contiguous_idx ^ (partition_strided_idx % 2);

    int element_contiguous = (tile_contiguous_idx * TileShape::kContiguous +
        permuted_partition_contiguous_within_tile *
            PartitionShape::kContiguous +
        permuted_vec_contiguous_within_partition) *
        kElementsPerAccess +
        (coord.contiguous() % kElementsPerAccess);

    int element_strided = vec_strided_idx;
    return element_contiguous + element_strided * stride_[0] * kFactor;
}
```



```
auto swizzle_layout_atom = composition(
    Swizzle<3,3,3>{},
    Layout<Shape <Shape <_2, _4, _2>, Shape <_8, _2, _2>>,
        Stride<Stride<_8, _64, _32>, Stride<_1, _16, _256>>>{});

auto smem_layout = tile_to_shape(swizzle_layout_atom, Shape<_128, _128>{});
```


EinFold

PyCuTe folding, as generalized transpose

- Mode permutation, grouping, ungrouping, replication, deletion.

CuTe / examples / einfold_test.py

```
>>> A = Layout(((2, 3), 5, 7), ((1, 2), 6, 30))
```

subscripts	einfold(subscripts, A)	
<code>ijk -> kji</code>	<code>(7, 5, (2, 3)):(30, 6, (1, 2))</code>	permute top-level modes
<code>ijk -> i(jk)</code>	<code>((2, 3), (5, 7)):(((1, 2), (6, 30)))</code>	group <code>j,k</code> into one mode
<code>(ab)c -> abc</code>	<code>(2, 3, 5, 7):(1, 2, 6, 30)</code>	descend into mode 0 — <code>k</code> unnamed, appended
<code>(ab)c -> (ba)c</code>	<code>((3, 2), 5, 7):((2, 1), 6, 30)</code>	reorder <i>inside</i> a nested mode
<code>i -> {i}</code>	<code>((2, 3), 5, 7),):(((1, 2), 6, 30),)</code>	a lone name binds the whole source
<code>ijk -> ik</code>	<code>((2, 3), 7):((1, 2), 30)</code>	drop the mode <code>j</code> binds
<code>ijk -> iki</code>	<code>((2, 3), 7, (2, 3)):(1, 2), 30, (1, 2))</code>	repeat — duplicate a mode

Better Tensor Algorithms

CuTe (CUDA Tensor) Framework

Examples

Various 4x8 layouts

2.4.2 Layout Examples

Having defined shapes, strides, and layouts, we now present examples that demonstrate CUTE layouts are a strict generalization of common flat N-dimensional layouts.

0	4	8	12	16	20	24	28
1	5	9	13	17	21	25	29
2	6	10	14	18	22	26	30
3	7	11	15	19	23	27	31

(a) Col-Major
(4, 8) : (1, 4)

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

(b) Row-Major
(4, 8) : (8, 1)

0	5	10	15	20	25	30	35
1	6	11	16	21	26	31	36
2	7	12	17	22	27	32	37
3	8	13	18	23	28	33	38

(c) Col-Major Padded
(4, 8) : (1, 5)

0	1	2	3	16	17	18	19
4	5	6	7	20	21	22	23
8	9	10	11	24	25	26	27
12	13	14	15	28	29	30	31

(d) Col-Major Interleave
(4, (4, 2)) : (4, (1, 16))

0	2	4	6	16	18	20	21
1	3	5	7	17	19	22	23
8	10	12	14	24	26	28	30
9	11	13	15	25	27	29	31

(e) Mixed
((2, 2), (4, 2)) : ((1, 8), (2, 16))

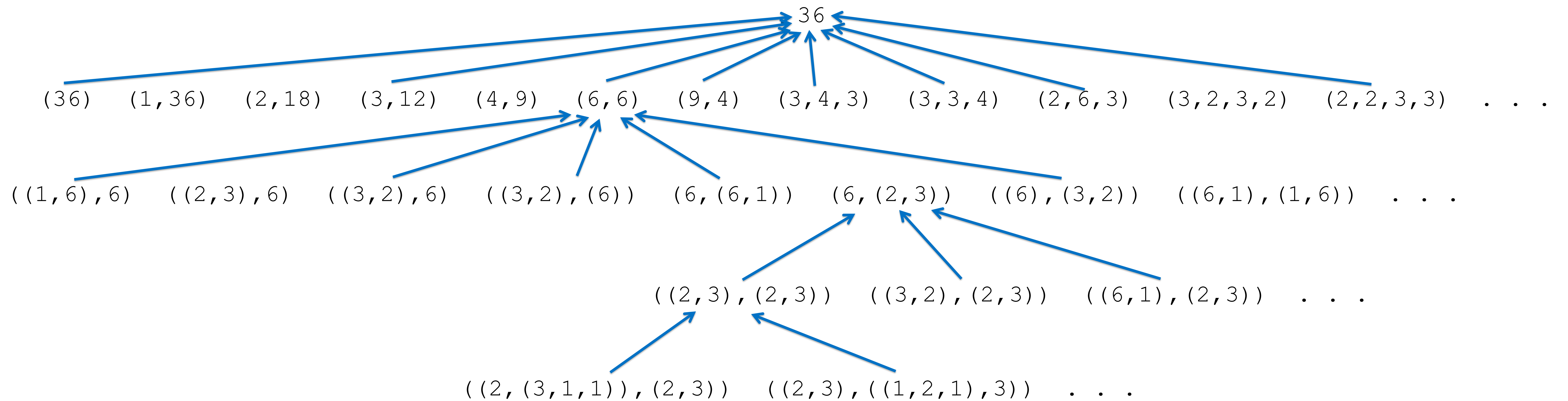
0	0	4	4	8	8	12	12
0	0	4	4	8	8	12	12
2	2	6	6	10	10	14	14
2	2	6	6	10	10	14	14

(f) Blocked Broadcast
((2, 2), (2, 4)) : ((0, 2), (0, 4))

Figure 3: Examples of layouts compatible with shape (4, 8) plotted as 4×8 matrices. These layouts use integer strides and hierarchical shapes to define functions useful for data tensors. The light line follows the general order of the layout offsets.

Layout Compatibility

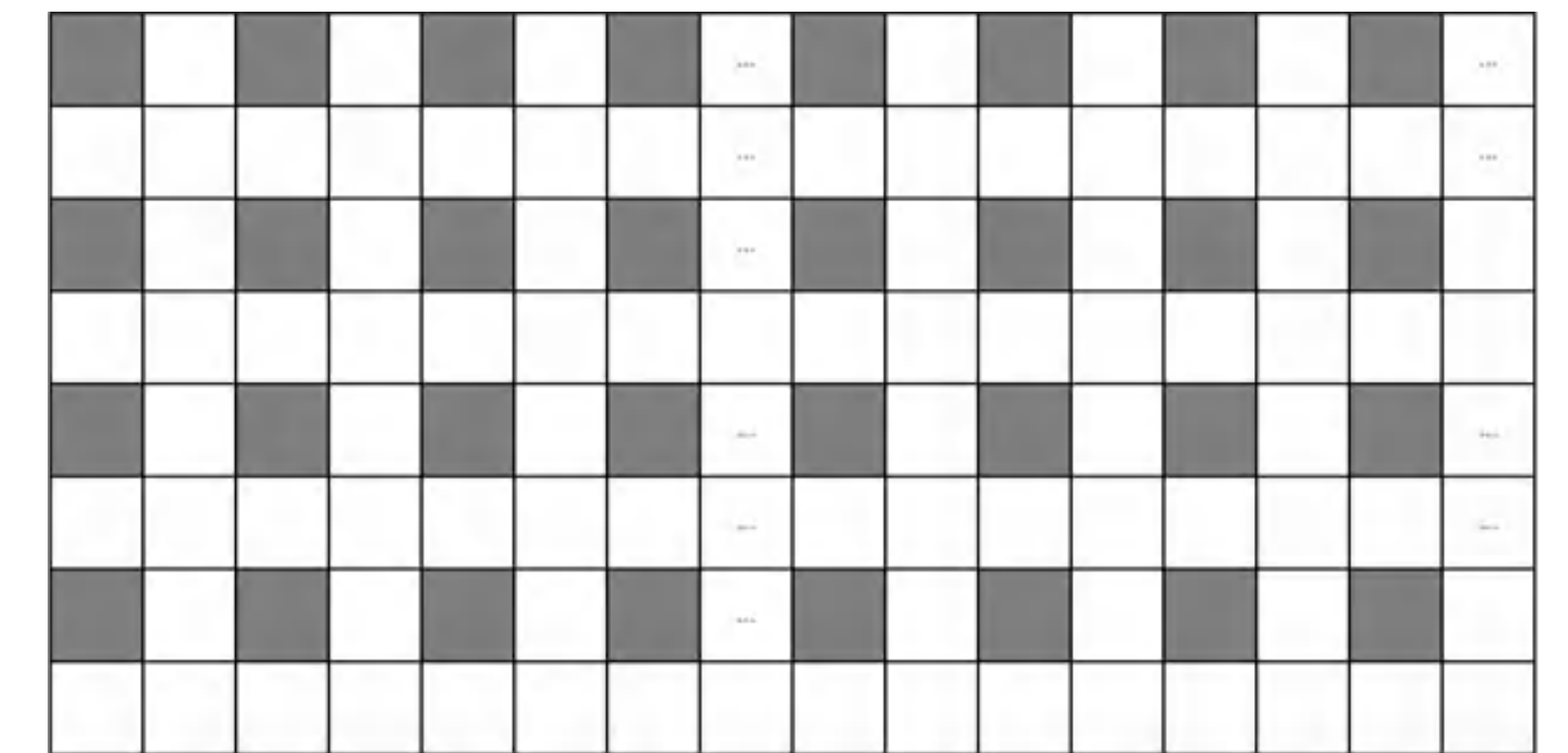
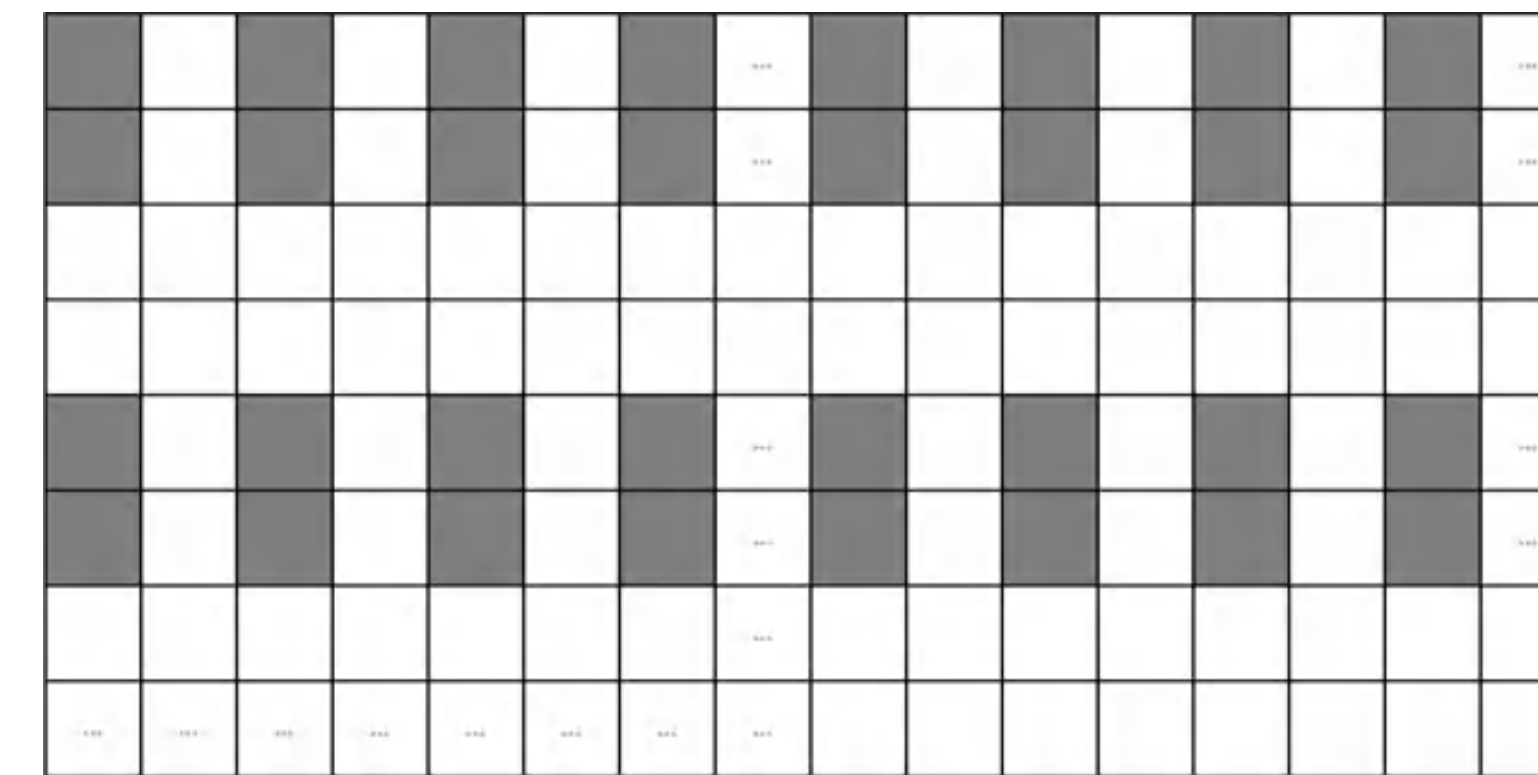
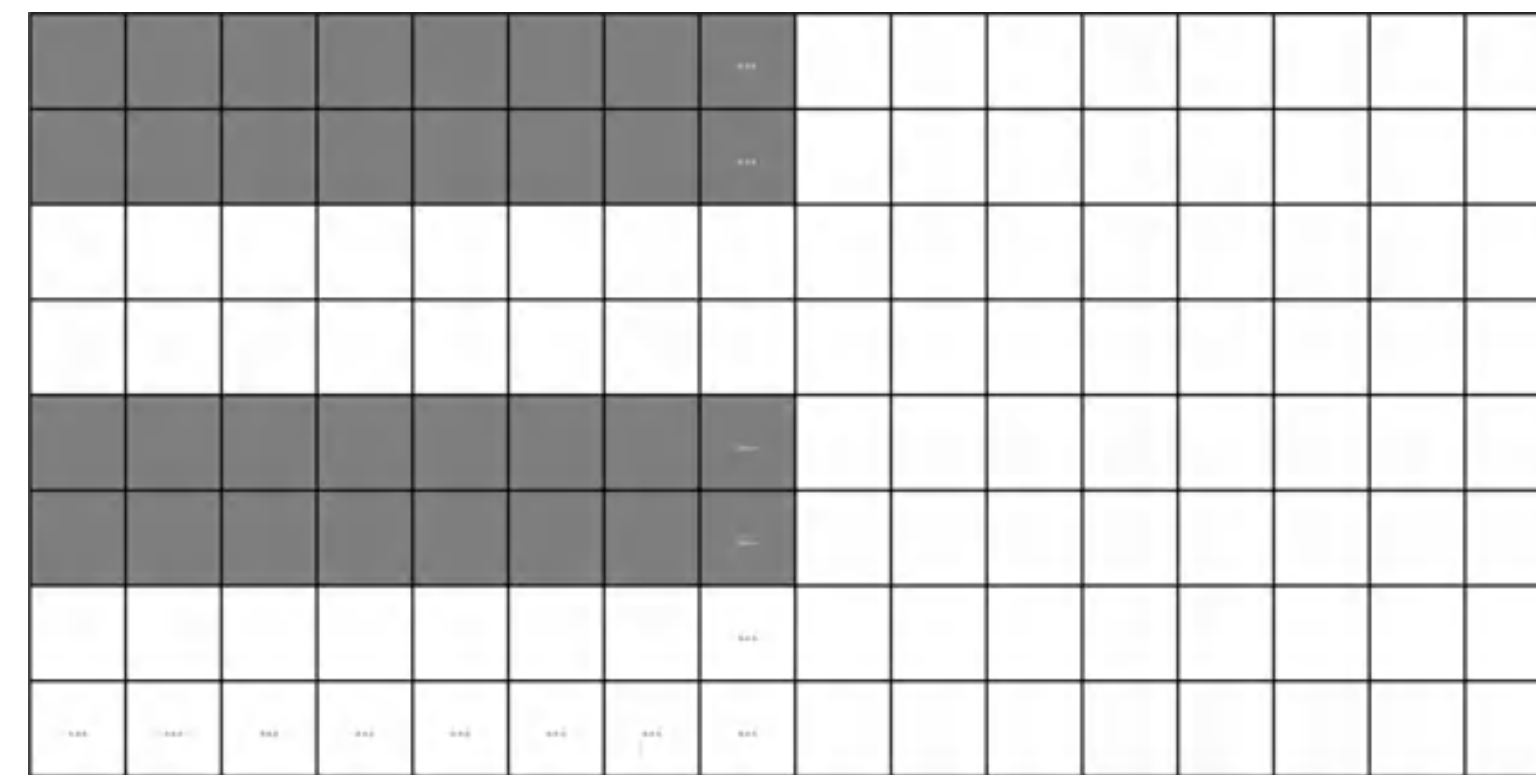
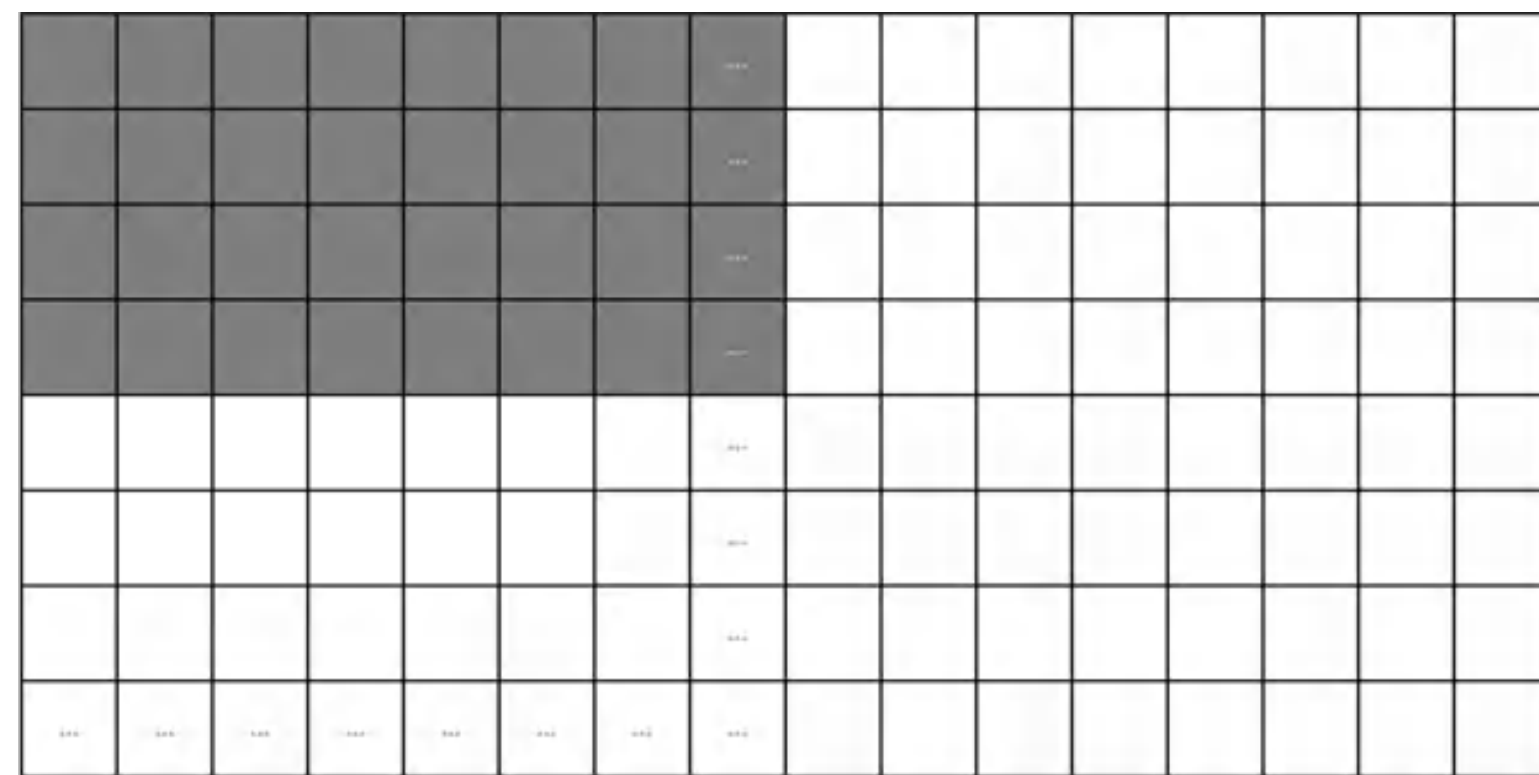
A poset of Shapes



Why do we want these different views?

Choose coordinates that fit the job

Many ways to tile a tensor into 4x8 subtensors



Many ways to construct a 4x8 layout

0	4	8	12	16	20	24	28
1	5	9	13	17	21	25	29
2	6	10	14	18	22	26	30
3	7	11	15	19	23	27	31

(a) Col-Major
(4, 8) : (1, 4)

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

(b) Row-Major
(4, 8) : (8, 1)

0	5	10	15	20	25	30	35
1	6	11	16	21	26	31	36
2	7	12	17	22	27	32	37
3	8	13	18	23	28	33	38

(c) Col-Major Padded
(4, 8) : (1, 5)

0	2	4	6	16	18	20	21
1	3	5	7	17	19	22	23
8	10	12	14	24	26	28	30
9	11	13	15	25	27	29	31

(e) Mixed
((2, 2), (4, 2)) : ((1, 8), (2, 16))

One copy interface:

```
cute::copy(src_4x8_tensor, dst_4x8_tensor);
```


Generic tensor copy

Copy is a rank-1 algorithm

```
def copy(src: Tensor, dst: Tensor) -> None:          # pycute.alg.ref.copy
    if size(src) != size(dst):
        raise ValueError(f"copy: size mismatch {size(src)} != {size(dst)}")

    for i in range(size(dst)):
        dst[i] = src[i]
```

No indexing overhead
for static shapes

Application	Source Layout	Destination Layout
1D Arrays	8 : 1	8 : 1
ND Arrays	(8, 2, 3) : (1, 16, 32)	(8, 2, 3) : (1, 16, 32)
Gather	(2, 3, 2) : (42, 1, 128)	12 : 1
Scatter	12 : 1	(2, 3, 2) : (42, 1, 128)
Broadcast	7 : 0	7 : 1
Constant	7 : 0	7 : 0
Transpose	(8, 3) : (1, 8)	(8, 3) : (3, 1)
Tensor Transpose	(8, (3, 5)) : (1, (57, 8))	(8, 15) : (1, 8)

Table 2: Applications of COPY with example source and destination layouts.

Tensor copy abstraction in action

Two versions of the same Hopper GEMM kernel

```
Layout C_layout = make_layout(make_shape(M,N), make_stride(1_s,M));
Tensor mC = make_tensor(ptr, C_layout);
```

```
// Loop over Tiles
for (int tile_num = bid; tile_num < size<1>(mC_tiled); tile_num += 132)
{
    // Tile and slice for tile_num (blockIdx)
    Tensor gC = local_tile(mC, make_tile(_128,_128), tile_num);

    // Tile and slice for MMA (threadIdx)
    ThrMMA thr_mma = mma.get_slice(threadIdx.x);
    Tensor tCgC = thr_mma.partition_C(gC);
    Tensor tCrC = thr_mma.make_fragment_C(tCgC);

    // ► Perform tile-wise product here

    // Epilogue
    copy(tCrC, tCgC);
}
```

```
// Loop over Tiles
for (int tile_num = bid_1; tile_num < (M / 128) * (N / 128); tile_num += 132)
{
    // ► Perform tile-wise product here

    // Epilogue
    int epilogue_tid_1 = ((threadIdx.x / 4) % 8);
    int epilogue_tid_2 = (threadIdx.x % 4);
    #pragma unroll
    for (int epilogue_m = 0; epilogue_m < 2; ++epilogue_m) {
        #pragma unroll
        for (int epilogue_n = 0; epilogue_n < 32; ++epilogue_n) {
            C[((((((((epilogue_n % 2) * M) + ((epilogue_n / 2) * (M * 8))) +
                (epilogue_m * 8)) +
                (((M * 2) * epilogue_tid_2) + epilogue_tid_1)) +
                (gwid_1 * 16)) +
                (math_group_id_1 * 64)) +
                (((tile_num / ((M + 127) / 128)) % ((N + 127) / 128)) * (M * 128)) +
                ((tile_num % ((M + 127) / 128)) * 128)))] =
                c_rf[((((epilogue_n / 2) * 4) + (epilogue_n % 2)) +
                    (epilogue_m * 2))];
        } // end-epilogue_n-loop
    } // end-epilogue_m-loop
} // end-tile_num-loop
```


COPY Deep Dive

Layout Analysis in Algorithms

Transforming Access Pattern of Generic Copy

Correctness is invariant under permutation

The reference picked one iteration order, what about other choices?

```
for i in range(size(dst)):  
    dst[P(i)] = src[P(i)]
```



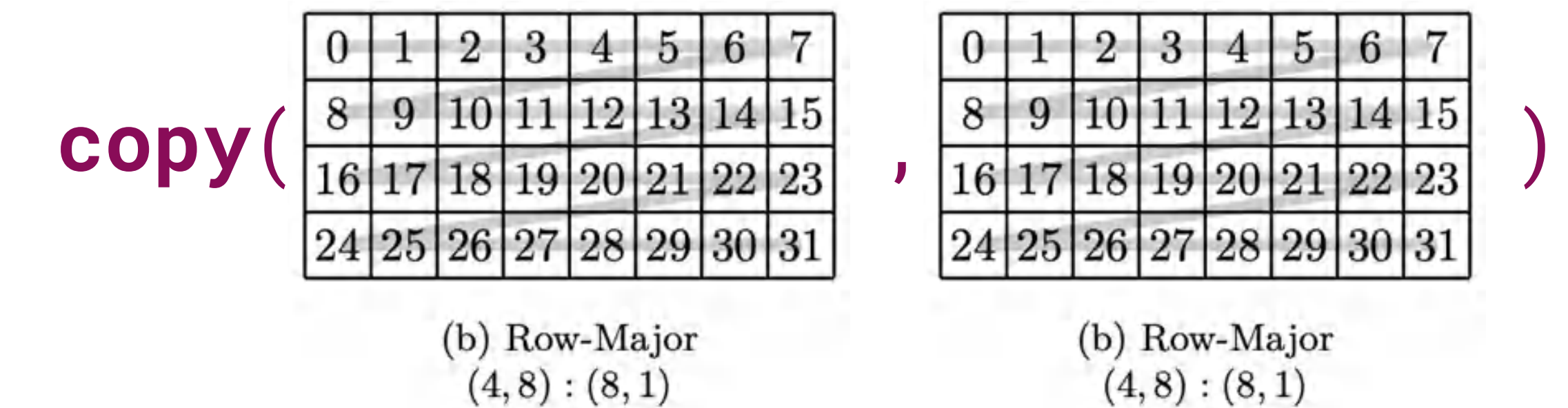
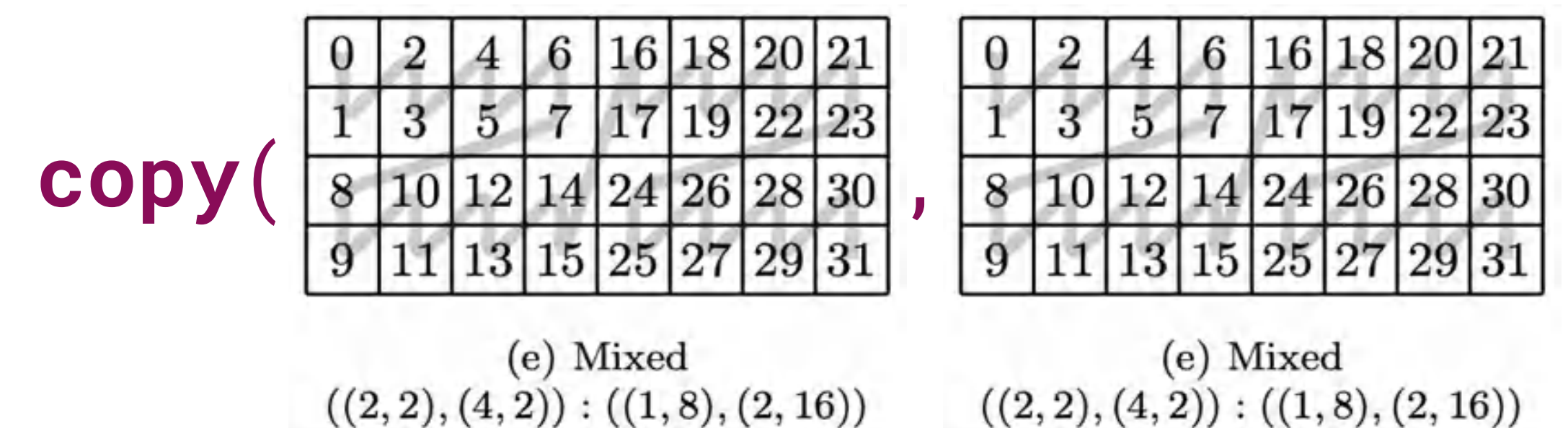
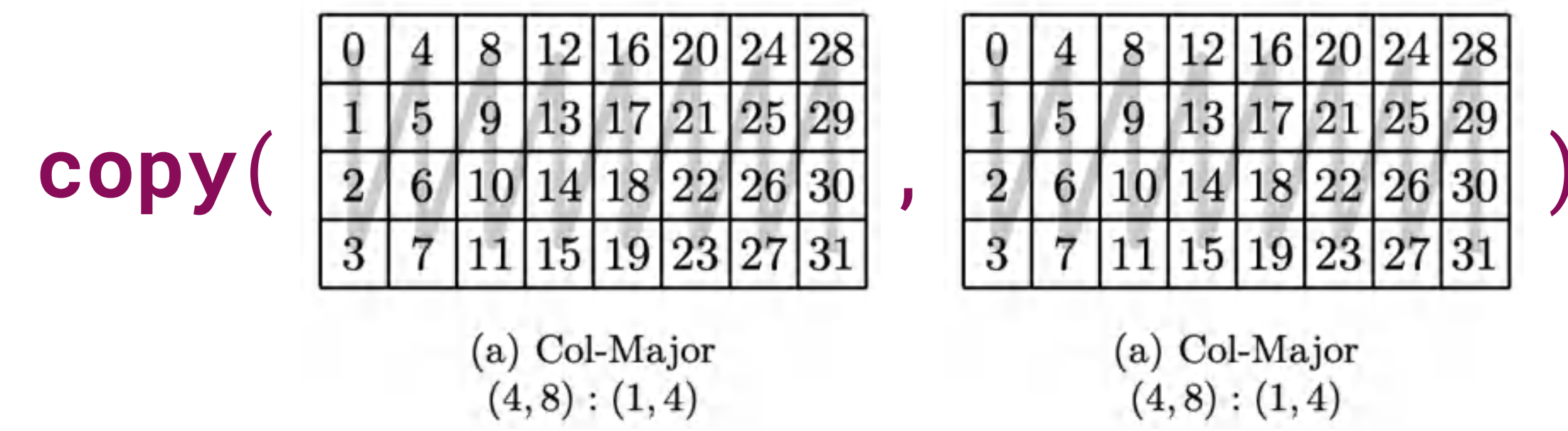
Alternative access patterns are just
permutations

$$P : \mathbb{Z} \rightarrow \mathbb{Z}$$

What permutation do we want?

In a perfect world, we'd convert every generic copy to a trivial flat copy

```
template <class T, int N>
void array_copy(std::array<T,N> const& src, std::array<T,N> & dst)
{
    for (int k = 0; k < size(dst); ++k) {
        dst[k] = src[k];
    }
}
```



How would we find this permutation?

Let's imagine the algebraic transformations we wish we could make

The assignment :

$$\text{dst}[i] = \text{src}[i]$$

is defined as :

$$\text{dst_ptr}[\text{LD}(i)] = \text{src_ptr}[\text{LS}(i)]$$

where LD and LS are layouts of dst and src.

Introducing a permutation P :

$$\text{dst_ptr}[\text{LD}(P(j))] = \text{src_ptr}[\text{LS}(P(j))]$$

we can re-associate to:

$$\text{dst_ptr}[(\text{LD} \circ P)(j)] = \text{src_ptr}[(\text{LS} \circ P)(j)]$$

How would we imagine finding this permutation?

Let's imagine the algebraic transformations we wish we could make

Given this:

$$\text{dst_ptr}[(LD \circ P)(j)] = \text{src_ptr}[(LS \circ P)(j)]$$

We'd be all set if :

$$I == LS \circ P$$

$$\text{and/or } I == LD \circ P$$

Wait, do Layouts compose?

Implying that we want the permutation:

$$P = LS^{-1}$$

$$\text{or } P = LD^{-1}$$

Wait, Layouts have inverses??

Key Idea #2: Treat Layouts Algebraically

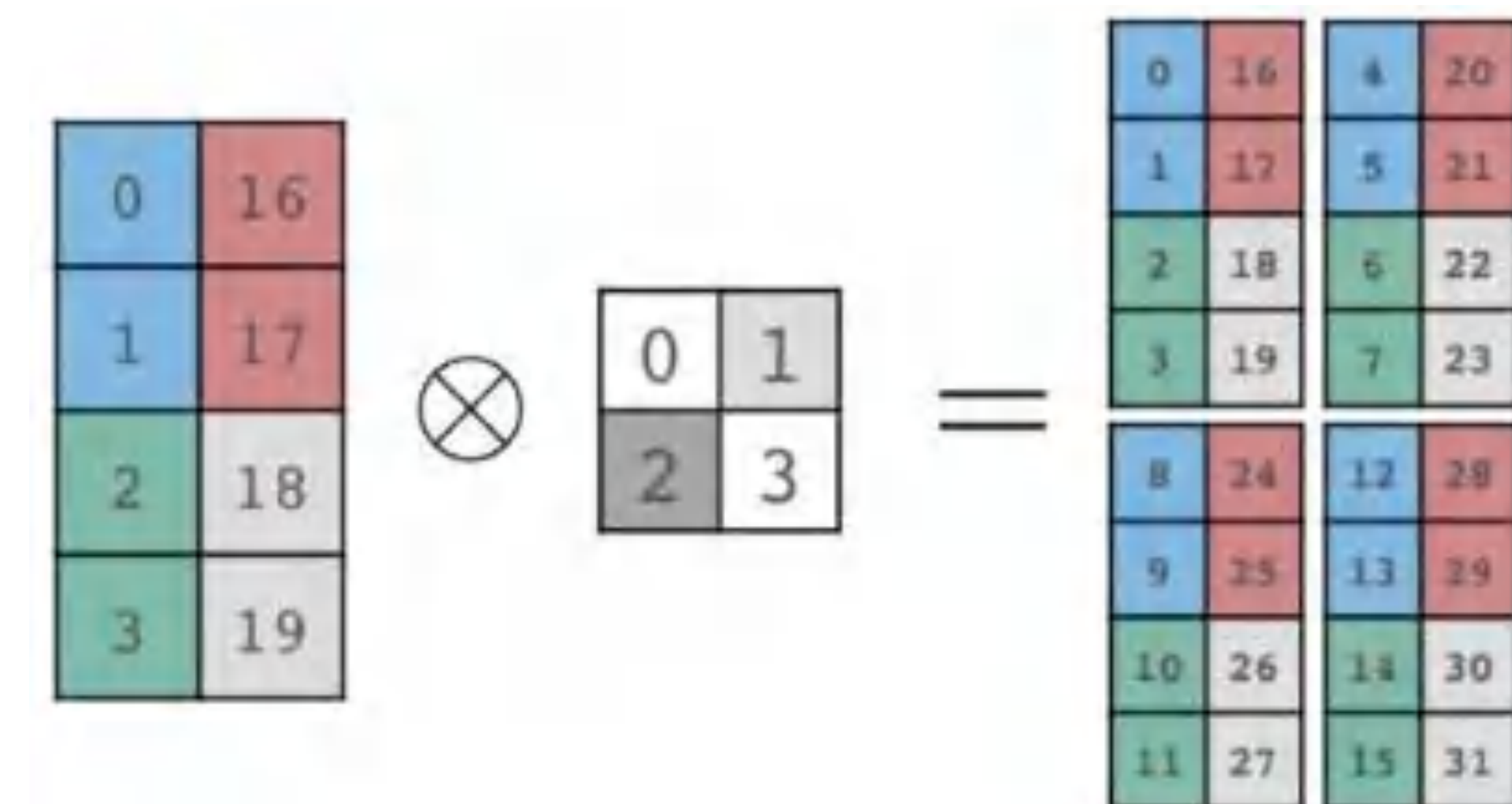
Operations on Layouts result in a Layout

Selection of Algebraic Primitives

`composition(LayoutA, LayoutB)` -> Layout
`concatenation(Layouts...)` -> Layout
`coalesce(Layout)` -> Layout
`right_inverse(Layout)` -> Layout
`left_inverse(Layout)` -> Layout
`nullspace(Layout)` -> Layout
`complement(Layout)` -> Layout

Used to build more complex transforms:

`logical_product(LayoutA, LayoutB)` -> Layout
`logical_divide (LayoutA, LayoutB)` -> Layout
`common_layout(LayoutA, LayoutB)` -> Layout



—————→ Example: $A \otimes B$

Every element of layout B is a layout A

Applications to COPY

Derive the best permutation!

- Derive a the best permutation from the SrcLayout and the DstLayout.
- Each stage applies a permutation and establishes one invariant.

Stage	Tool	What it establishes
1. Common domain	<code>greatest_common_domain</code>	a tiler admissible for both layouts
2. Nullspace	<code>nullspace</code>	no stride-0 coordinate writes an address twice
3. Vectorization	<code>right_inverse</code> , <code>coalesce</code>	<code>dst</code> in memory order, innermost run <code>v:1</code> in both
4. Dispatch	<code>slicing</code>	each <code>V</code> -wide slice handed to a lower-level copy

- The result of every stage is still consistent with COPY.
 - Never *less correct*, worst case it degrades to the reference copy.


```
def copy(src: Tensor, dst: Tensor) -> None:
    """COPY with Layout Analysis"""
    if size(src) != size(dst):
        raise ValueError(f"copy: size mismatch {size(src)} != {size(dst)}")

    # Extract the domain-compatible portions

    common_domain = greatest_common_domain(src, dst)
    src_c = logical_divide(src, common_domain) # (Compat, InCompat)
    dst_c = logical_divide(dst, common_domain) # (Compat, InCompat)

    # Account for nullspaces of dst and src within compatible portions

    null_dst = nullspace(dst_c.layout[0])
    src_n = logical_divide[0](src_c, null_dst) # ((Null, NonNull), InCompat)
    dst_n = logical_divide[0](dst_c, null_dst) # ((Null, NonNull), InCompat)

    if dst_n.layout[0][0](size(null_dst) - 1) != 0:
        raise ValueError(f"Sanity: nullspace definition error")
    if src_n.layout[0][0](size(null_dst) - 1) != 0:
        raise ValueError(f"Write-after-write error detected in dst: {dst}")

    # Slice the common stride-0 modes out
    src_n = src_n[(0, None), None] # (NonNull, Incompat)
    dst_n = dst_n[(0, None), None] # (NonNull, Incompat)

    # Vectorization of the compatible portions

    inv_dst = right_inverse(dst_n.layout[0])
    src_v = logical_divide[0](src_n, inv_dst) # ((Id, NonId), Incompat)
    dst_v = logical_divide[0](dst_n, inv_dst) # ((Id, NonId), Incompat)

    # This section is technically instruction-aware,
    # we are looking for a common contiguous run (vector) in src and dst
    # but other instructions may look for other patterns completely
    try:
        vec_size = 1
        src_dst_v = coalesce(src_v.layout[0][0])[0]
        if stride(src_dst_v) == 1:
            vec_size = size(src_dst_v)
    except TypeError:
        pass

    src_v = logical_divide(src_v, vec_size) # (V:1, Rest)
    dst_v = logical_divide(dst_v, vec_size) # (V:1, Rest)

    # Slice and dispatch to an optimized/vectorized memcpy

    for i in range(size[1](dst_v)):
        memcpy(src_v[:, i], dst_v[:, i])
```

COPY ipython notebook tutorial and discussion

CuTe / examples / algorithms / **copy.ipynb**

 ccecka Apply ModeOp algebra operations to COPY algorithm and tutorial
2e12e57 · last month
History

Preview Code Blame 1345 lines (1345 loc) · 105 KB
Raw

COPY in Layout Algebra

This notebook walks through the **COPY** algorithm from Cecka, *CuTe Layout Representation and Algebra* ([arXiv:2603.02298](https://arxiv.org/abs/2603.02298), §2.6.1), implemented in PyCuTe.

- Applications** — the same `copy(src, dst)` with different layouts realizes memcpy, gather/scatter, broadcast, transpose, ...
- Optimizations** — how `pycute.alg.copy` reshapes that loop with `greatest_common_domain`, `nullspace`, and `right_inverse`.


```

def copy(src: Tensor, dst: Tensor) -> None:
    """COPY with Layout Analysis"""
    if size(src) != size(dst):
        raise ValueError(f"copy: size mismatch {size(src)} != {size(dst)}")

    # Extract the domain-compatible portions

    common_domain = greatest_common_domain(src, dst)
    src_c = logical_divide(src, common_domain) # (Compat, InCompat)
    dst_c = logical_divide(dst, common_domain) # (Compat, InCompat)

    # Account for nullspaces of dst and src within compatible portions

    null_dst = nullspace(dst_c.layout[0])
    src_n = logical_divide[0](src_c, null_dst) # ((Null, NonNull), InCompat)
    dst_n = logical_divide[0](dst_c, null_dst) # ((Null, NonNull), InCompat)

    if dst_n.layout[0][0](size(null_dst) - 1) != 0:
        raise ValueError(f"Sanity: nullspace definition error")
    if src_n.layout[0][0](size(null_dst) - 1) != 0:
        raise ValueError(f"Write-after-write error detected in dst: {dst}")

    # Slice the common stride-0 modes out
    src_n = src_n[(0, None), None] # (NonNull, Incompat)
    dst_n = dst_n[(0, None), None] # (NonNull, Incompat)

    # Vectorization of the compatible portions

    inv_dst = right_inverse(dst_n.layout[0])
    src_v = logical_divide[0](src_n, inv_dst) # ((Id, NonId), Incompat)
    dst_v = logical_divide[0](dst_n, inv_dst) # ((Id, NonId), Incompat)

    # This section is technically instruction-aware,
    # we are looking for a common contiguous run (vector) in src and dst
    # but other instructions may look for other patterns completely
    try:
        vec_size = 1
        src_dst_v = coalesce(src_v.layout[0][0])[0]
        if stride(src_dst_v) == 1:
            vec_size = size(src_dst_v)
    except TypeError:
        pass

    src_v = logical_divide(src_v, vec_size) # (V:1, Rest)
    dst_v = logical_divide(dst_v, vec_size) # (V:1, Rest)

    # Slice and dispatch to an optimized/vectorized memcpy

    for i in range(size[1](dst_v)):
        memcpy(src_v[:, i], dst_v[:, i])

```

Common Domain for admissible permutations

Nullspace elimination to remove redundancy and detect WAW

"Vectorization"

Dispatch and lowering
(e.g. parallelize here)

Greatest Common Domain

Stage 1: A subshape both tensors agree on

- Each stage permutes SrcTensor and DstTensor consistently.
 - Each permutation is always applied to BOTH.
- But some shapes admit no common non-trivial tilers.
 - e.g. (5,7) and (7,5) are *coprime* and cannot both be "divided" by any non-trivial Layout.
- `greatest_common_domain` (GCD) finds the largest subshape of both.
 - This subshape can be consistently permuted.
 - The GCD divides both Layouts into (Compat, Incompat)

case	src	dst	common domain	Compat	InCompat
transpose	(4, 8) : (8, 1)	(4, 8) : (1, 4)	(4, 8) : (1, 4)	32	1
gather	(2, 3) : (4, 1)	6 : 1	(2, 3) : (1, 2)	6	1
subdomain	(4, 3, 5) : (1, 7, 42)	(6, 10) : (1, 9)	(2, 5) : (1, 12)	10	6
coprime	(5, 7) : (7, 1)	(7, 5) : (5, 1)	(1,) : (0,)	1	35

Nullspace

Stage 2: Redundant writes and write-after-write

- `nullspace` is the sublayout of coordinates `L` sends to offset 0.

```
>>> dst = Layout((3, 5), (1, 0)); nullspace(dst)
5:3                                     # 15 coordinates over 3 addresses, 5 per address
```

- Divide both `Compat` modes by this to get `(Null, NonNull)`, then inspect:

DstNull	SrcNull	Verdict
5:0	5:0	Accept
5:0	5:X	Reject – write-after-write

- A `src` with stride-0s across `dst`'s `nullspace` works: **one value is being copied to one value.**
- A `src` with stride-Xs across `dst`'s `nullspace` fails: **many values are racing into one value.**

Vectorization

Stage 3: Determine common vectors (w.r.t. instructions)

- `memcpy/array_copy` is simply copy of `src` layout `N:1` to `dst` layout `N:1`.
- What permutation takes `L` there?
 - `L ○ P => N:1` that is `P = right_inverse(L)`
- So apply that to BOTH tensors and see "how much the other agrees".

```
# Vectorization of the compatible portions

inv_dst = right_inverse(dst_n.layout[0])
src_v = logical_divide[0](src_n, inv_dst) # ((Id, NonId), Incompat)
dst_v = logical_divide[0](dst_n, inv_dst) # ((Id, NonId), Incompat)

# This section is technically instruction-aware,
# we are looking for a common contiguous run (vector) in src and dst
# but other instructions may look for other patterns completely
try:
    vec_size = 1
    src_dst_v = coalesce(src_v.layout[0][0])[0]
    if stride(src_dst_v) == 1:
        vec_size = size(src_dst_v)
except TypeError:
    pass

src_v = logical_divide(src_v, vec_size) # (V:1, Rest)
dst_v = logical_divide(dst_v, vec_size) # (V:1, Rest)
```


Dispatch

Stage 4: Consume via slicing or parallelizing

- Stage 3 left us with $(V:1, \text{Rest})$
 - Mode-0 is the **instruction** (or whatever subtensor will be lowered to the next algorithm)
 - Mode-1 is the **schedule** of those instructions

```
for i in range(size[1](dst_v)):          # pycute.alg.copy
    _memcpy(src_v[:, i], dst_v[:, i])    # _memcpy is pycute.alg.ref.copy
```

- The slices are independent...

```
_memcpy(src_v[:, thread_id], dst_v[:, thread_id])
```

- and now we've written a `cooperative_copy`

COPY with Layout Analysis

Permutation of the iteration space

`greatest_common_domain` → what can be permuted together

`nullspace` → which writes are redundant, and which race

`right_inverse + coalesce` → memory order, then the width

`slicing` → one instruction per slice, looped or threaded

`pycute/alg/copy.py` · `examples/algorithms/copy.ipynb`

Generic GEMM

Another trivial algorithm

```
def gemm(A, B, C):                                # pycute.alg.ref.gemm
    for l in range(size[2](C)):                   # batch      L
        for k in range(size[1](A)):               # reduction  K
            for n in range(size[1](C)):           # column     N
                for m in range(size[0](C)):       # row        M
                    C[m,n,l] += A[m,k,l] * B[n,k,l]
```

- COPY is a 1D algorithm, GEMM is a 3D/4D algorithm.

Generic GEMM

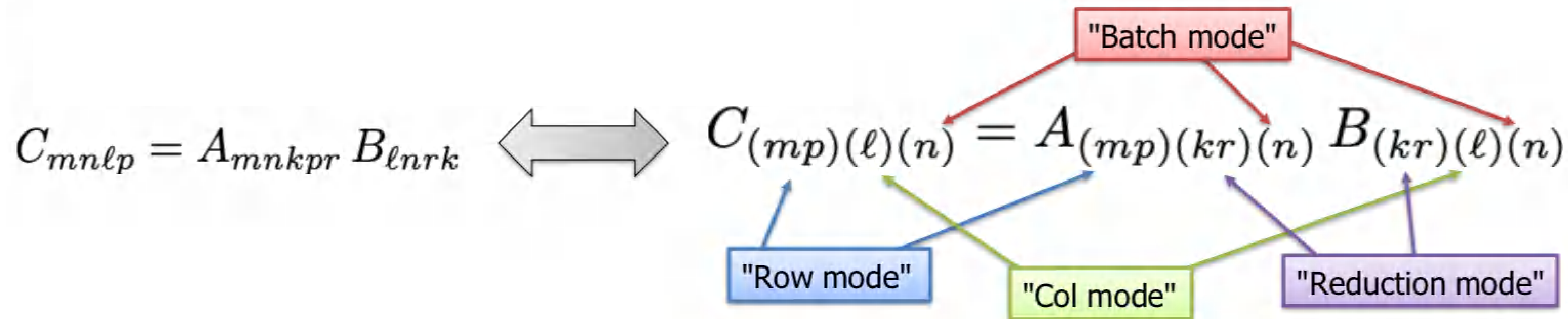
With loads of applications

	A	B	C
NT GEMM	$(M, K) : (1, lda)$	$(N, K) : (1, ldb)$	$(M, N) : (1, ldc)$
TN GEMM	$(M, K) : (lda, 1)$	$(N, K) : (ldb, 1)$	$(M, N) : (1, ldc)$
BLIS GEMM	$(M, K) : (dma, dka)$	$(N, K) : (dnb, dkb)$	$(M, N) : (dmc, dnc)$
GETT	$((M_1, M_2), K) : ((1, W), X)$	$(N, K) : (K, 1)$	$((M_1, M_2), N) : ((1, Y), Z)$
grouped GEMM	$(M, K, G) : (1, M, M * K)$	$(N, K, G) : (1, N, 0)$	$(M, N, G) : (1, M, M * N)$
CONV	$((N, (Z, P, Q)), (C, (T, R, S)))$	$(K, (C, (T, R, S)))$	$((N, (Z, P, Q)), K)$

- COPY operates on logical size-N, GEMM operates on logical $M \times K \times L$ and logical $N \times K \times L$.
- Instead of saying B is "transposed" or "non-transposed", tell me whether B is "N-major" or "K-major" or neither.
- CONV is interesting...

General Tensor Folding

All GETTs are GEMMs



All tensor contractions map to a canonical contraction: **Batched-GEMM**

- By grouping modes by type and defining **multi-modes**:
 - m-modes** or "row modes" appear in C & A & $!B$
 - n-modes** or "column modes" appear in C & $!A$ & B
 - k-modes** or "reduction modes" appear in $!C$ & A & B
 - p-modes** or "batch modes" appear in C & A & B

$$C_{\hat{m}\hat{n}\hat{p}} = A_{\hat{m}\hat{k}\hat{p}} B_{\hat{k}\hat{n}\hat{p}}$$

Shi, Niranjan, Anandkumar, **Cecka**. *Tensor Contractions with Extended BLAS Kernels on CPU and GPU*, IEEE 2016.

Cecka. *Low communication FMM-accelerated FFT on GPUs*, SuperComputing 2017.

EinSum

PyCuTe example utility

`examples/einsum.py` classifies the labels, folds the operands, calls `gemm` :

subscripts	M	N	K	L	
<code>mk,nk->mn</code>	<code>m</code>	<code>n</code>	<code>k</code>	<code>-</code>	GEMM
<code>mk,nk->nm</code>	<code>m</code>	<code>n</code>	<code>k</code>	<code>-</code>	Table 3's NTT — a relabeling, not a transpose
<code>bmk,bnk->bm</code>	<code>m</code>	<code>n</code>	<code>k</code>	<code>b</code>	batched GEMM
<code>ijp,np->ijn</code>	<code>ij</code>	<code>n</code>	<code>p</code>	<code>-</code>	GETT — two row modes
<code>stupr,qtru->stqp</code>	<code>sp</code>	<code>q</code>	<code>ur</code>	<code>t</code>	the contraction above
<code>ij,j->i</code>	<code>i</code>	<code>-</code>	<code>j</code>	<code>-</code>	gemv — no column mode
<code>i,i-></code>	<code>-</code>	<code>-</code>	<code>i</code>	<code>-</code>	dot — only the <code>k</code> loop survives
<code>i,j->ij</code>	<code>i</code>	<code>j</code>	<code>-</code>	<code>-</code>	ger — outer product, <code>K = 1</code>
<code>i,i->i</code>	<code>-</code>	<code>-</code>	<code>-</code>	<code>i</code>	Hadamard — <code>size(i)</code> independent <code>1x1x1</code> GEMMs
<code>mn,->nm</code>	<code>mn</code>	<code>-</code>	<code>-</code>	<code>-</code>	scaled transpose — <code>B</code> is rank-0

CONV

Im2col layout transformation, more than a fold

```
def _im2col_layout(A : Layout, *, stencil, shape_zpq,
                  lower_dhw, stride_dhw, lower_trs, stride_trs):
    # Both halves compose the same spatial sublayout; only the tiler differs.
    tiler_zpq = tuple(Layout(s, d) for s, d in zip(shape_zpq, stride_dhw))
    tiler_trs = tuple(Layout(s, d) for s, d in zip(stencil, stride_trs))
    A_dhw = A[1:-1] # the spatial modes

    layout = make_layout([
        make_layout([A[0], composition(A_dhw, tiler_zpq)]), # (N,(Z,P,Q))
        make_layout([A[-1], composition(A_dhw, tiler_trs)]), # (C,(T,R,S))
    ])

    origin = A(ArithTuple(0, *lower_trs, 0) - ArithTuple(0, *lower_dhw, 0))
    return layout, origin
```


Im2col example

Just a Layout transform

A single 4x4 image, one channel, a 2x2 stencil:

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15

```
>>> print(act.layout)    # (N,H,W,C)
(1, 4, 4, 1):(16, 4, 1, 1)
```

	0	1	2	3
0	0	4	1	5
1	4	8	5	9
2	8	12	9	13
3	1	5	2	6
4	5	9	6	10
5	9	13	10	14
6	2	6	3	7
7	6	10	7	11
8	10	14	11	15

```
>>> print(im2col(act, (2, 2)).layout)
((1, (3, 3)), (1, (2, 2)))
:((16, (4, 1)), (1, (4, 1)))
```


Im2col dilation

Striding the stencil

Mode 1 of the result is the stencil footprint. Drawn over the activation:

	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	7	8	9	10	11	12	13
2	14	15	16	17	18	19	20
3	21	22	23	24	25	26	27
4	28	29	30	31	32	33	34
5	35	36	37	38	39	40	41
6	42	43	44	45	46	47	48

	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	7	8	9	10	11	12	13
2	14	15	16	17	18	19	20
3	21	22	23	24	25	26	27
4	28	29	30	31	32	33	34
5	35	36	37	38	39	40	41
6	42	43	44	45	46	47	48

```
>>> im2col(A, (3, 3)).layout[1]  
(1, (3, 3)):(1, (7, 1))
```

nine contiguous taps

```
>>> im2col(A, (3, 3),  
...        stride_trs=(2, 2)).layout[1]  
(1, (3, 3)):(1, (14, 2))
```

nine taps, every other element

Im2col traversal stride

Striding the starts of the stencil

Mode **0** of the result is where the windows start. Same activation:

	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	7	8	9	10	11	12	13
2	14	15	16	17	18	19	20
3	21	22	23	24	25	26	27
4	28	29	30	31	32	33	34
5	35	36	37	38	39	40	41
6	42	43	44	45	46	47	48

```
>>> im2col(A, (3, 3)).layout[0]  
(1, (5, 5)):(49, (7, 1))
```

5x5 = 25 windows

	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	7	8	9	10	11	12	13
2	14	15	16	17	18	19	20
3	21	22	23	24	25	26	27
4	28	29	30	31	32	33	34
5	35	36	37	38	39	40	41
6	42	43	44	45	46	47	48

```
>>> im2col(A, (3, 3),  
...         stride_dhw=(2, 2)).layout[0]  
(1, (3, 3)):(49, (14, 2))
```

3x3 = 9 windows

Im2col OOB and padding

Via OOB Accessor

Ptr o [Offset <- (N, H, W, C)] o Pred(N,H,W,C) o [(n, h, w, c) <- (M,K)]

```
class OutOfBoundsAccessor(Accessor):
    def __getitem__(self, coord):
        extents = shape(self.tensor)
        # ArithTuple addition is elementwise to any depth, so the origin accumulates
        # into a hierarchical coordinate as readily as a flat one.
        crd = idx2crd(ArithTuple(self.origin) + ArithTuple(coord), extents)
        return self.tensor[crd] if in_bounds(crd, extents) else self.fill
```

Because `im2col_coord`'s layout delivers **coordinates**, the accessor can bounds-check *before* it reads — TMA's out-of-bounds fill, in software.

Padding costs one comparison per element and zero memory, so a padded convolution runs through an unmodified GEMM.

32 of 81 cells here are fill (40%), and not one is stored.

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	1	2	0	4	5
1	0	0	0	1	2	3	4	5	6
2	0	0	0	2	3	0	5	6	0
3	0	1	2	0	4	5	0	7	8
4	1	2	3	4	5	6	7	8	9
5	2	3	0	5	6	0	8	9	0
6	0	4	5	0	7	8	0	0	0
7	4	5	6	7	8	9	0	0	0
8	5	6	0	8	9	0	0	0	0

3x3 image, 3x3 stencil, pad 1.
Grey cells read `fill = 0`.

Implicit GEMM

This matters

- Well known that materializing the `im2col` matrix is **bad strategy**
 - Costs a lot in **memory and bandwidth**

	4x4 , 2x2 , C=1	56x56 , 3x3 , C=64 , pad 1
activation	16	$56 \cdot 56 \cdot 64 = 200\,704$
im2col matrix	36 (2.25x)	$56 \cdot 56 \cdot 64 \cdot 9 = 1\,806\,336$ (9x)

- The im2col matrix is just another layout
 - The extra reads still happen, but hit cache.
 - The im2col matrix layout can still be **tilled, permuted, sliced** – everything that an optimized GEMM needs.
- The Implicit GEMM is a **change of view**, not a change of algorithm.



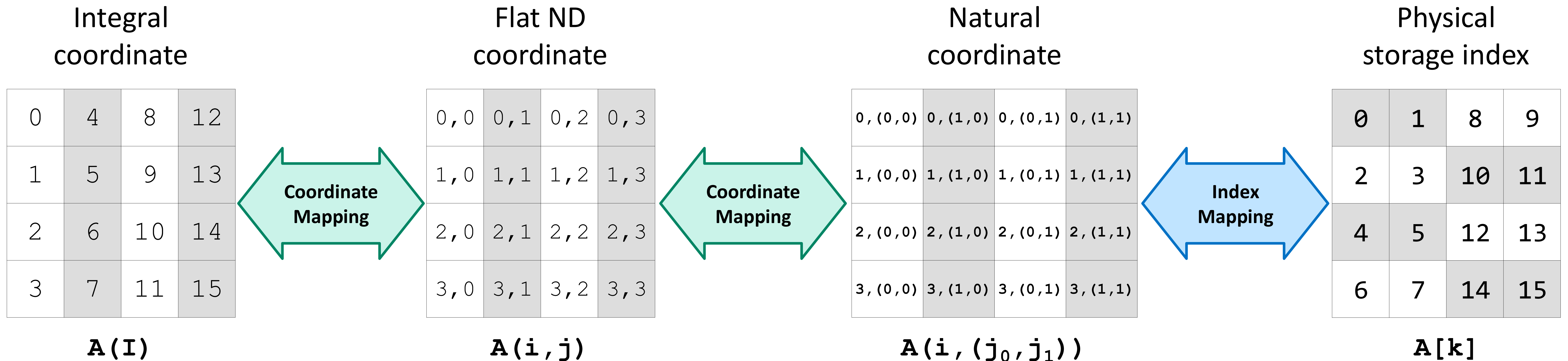
Abstracted Strides

Layout Mappings

Coordinates and Indices

Shape: $(4, (2, 2))$

Stride: $(2, (1, 8))$



Shape defines the *coordinate* mappings

$$I \Leftrightarrow (i, j) \Leftrightarrow (i, (j_0, j_1))$$

Stride defines the *index* mapping

$$(i, (j_0, j_1)) \rightarrow [k]$$

Shape is a function, Stride is a function

Layout is a functional composition

$$\mathbf{L} = D \circ S$$

$$S: Z \leftrightarrow \mathbb{Z}_S$$

idx2crd, crd2idx

$$D: \mathbb{Z}_S \rightarrow \mathcal{D}$$

inner_product

```
def __call__(self, *crd: Coord) -> StrideScalar:  
    return inner_product(idx2crd(crd, self.shape), self.stride)
```


What is a Stride?

Doesn't need much

$$\text{inner_product}(c, D) = \sum_i c_i d_i$$

```
def inner_product(a: Coord, b: Stride) -> StrideScalar:  
    return sum(x*y for x,y in zip_leaves(a,b))
```

- The only operations a StrideScalar must support are
 - * by an integer coordinate
 - + with other StrideScalars

$\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{Z}^S, \mathbb{F}$

Coordinate Strides

Just another StrideScalar

A uniquely useful integer-semimodule is $(\mathbb{Z}^S, +, \cdot)$ with $\mathbb{Z}^S$ the set of all `HTuple`($\mathbb{Z}$) congruent to S . For instance, the basis elements of rank-2 arithmetic tuples form an integer-semimodule:

$$e_0 = (1, 0), \quad e_1 = (0, 1), \quad \mathbb{Z}^{(*,*)} = \{a \cdot e_0 + b \cdot e_1 \mid a, b \in \mathbb{Z}\}$$

with integer scaling and addition defined element-wise:

$$a \cdot e_0 + b \cdot e_1 = (a, b)$$

```
>>> ArithTuple(1,2,(3,4)) + (7,8,(9,10))    # Arithmetic tuples
(8,10,(12,14))
>>> E(0) == ArithTuple(1, 0)                # a unit basis element
True
>>> 3*E(0) + 5*E(1)                          # a coordinate sum
(3, 5)
>>> ArithTuple(0, 0) == 0                    # int 0 is the identity, at any depth
True
```

Python	Description	String
<code>E()</code>	1	1
<code>E(0)</code>	$(1, 0, \dots)$	1@0
<code>E(1)</code>	$(0, 1, 0, \dots)$	1@1
<code>E(0,0)</code>	$((1, 0, \dots), 0, \dots)$	1@0@0
<code>E(0,1)</code>	$((0, 1, 0, \dots), 0, \dots)$	1@1@0
<code>E(1,0)</code>	$(0, (1, 0, \dots), 0, \dots)$	1@0@1
<code>E(1,1)</code>	$(0, (0, 1, 0, \dots), 0, \dots)$	1@1@1

Coordinate Strides

Layouts/Tensors produce coordinates

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)	(0,5)	(0,6)	(0,7)
(1,0)	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)	(1,7)
(2,0)	(2,1)	(2,2)	(2,3)	(2,4)	(2,5)	(2,6)	(2,7)
(3,0)	(3,1)	(3,2)	(3,3)	(3,4)	(3,5)	(3,6)	(3,7)

(a) Identity Coordinate
 $(4, 8) : (e_0, e_1)$

(0,0)	(1,0)	(2,0)	(3,0)	(0,6)	(1,6)	(2,6)	(3,6)
(0,1)	(1,1)	(2,1)	(3,1)	(0,7)	(1,7)	(2,7)	(3,7)
(0,2)	(1,2)	(2,2)	(3,2)	(0,8)	(1,8)	(2,8)	(3,8)
(0,3)	(1,3)	(2,3)	(3,3)	(0,9)	(1,9)	(2,9)	(3,9)

(b) Transposed Block Coordinate
 $(4, (4, 2)) : (e_1, (e_0, 6e_1))$

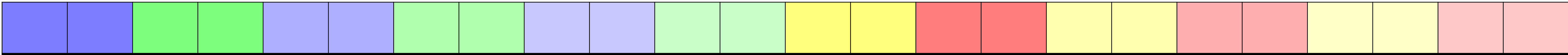
```
>>> from pycute import *
>>> from pycute.util import *
>>>
>>> A = Tensor(ImplicitAccessor(0), Layout((4,(4,2)), (E(1),(E(0),6*E(1)))))
>>> print_table(A)
{0} o (4, (4, 2)):(1@1, (1@0, 6@1))
+-----+-----+-----+-----+-----+-----+-----+-----+
| 0   | 1@0   | 2@0   | 3@0   | 6@1 | (1,6) | (2,6) | (3,6) |
+-----+-----+-----+-----+-----+-----+-----+
| 1@1 | (1,1) | (2,1) | (3,1) | 7@1 | (1,7) | (2,7) | (3,7) |
+-----+-----+-----+-----+-----+-----+-----+
| 2@1 | (1,2) | (2,2) | (3,2) | 8@1 | (1,8) | (2,8) | (3,8) |
+-----+-----+-----+-----+-----+-----+-----+
| 3@1 | (1,3) | (2,3) | (3,3) | 9@1 | (1,9) | (2,9) | (3,9) |
+-----+-----+-----+-----+-----+-----+-----+
>>>
>>> B = A[2,:]
>>> print_table(B)
{2@1} o ((4, 2),):(1@0, 6@1),)
+-----+-----+-----+-----+-----+-----+-----+
| 2@1 | (1,2) | (2,2) | (3,2) | 8@1 | (1,8) | (2,8) | (3,8) |
+-----+-----+-----+-----+-----+-----+-----+
```


Why Coordinate Layouts/Tensors?

- Predication
 - Tile/partition an identity_tensor the same way you tile/partition your data
 - Then compare coordinates against the problem bounds
- TMA Tensors
 - TMA consumes coordinates, not pointers/addresses.
 - Produce these with a layout/tensor instead of tracking by hand.
- Tilers
 - Use on the RHS of composition for more versatile tiling, this one is interesting...

Why is this important?

Do we need coordinate layouts?



((2, 3))

((1, 4))

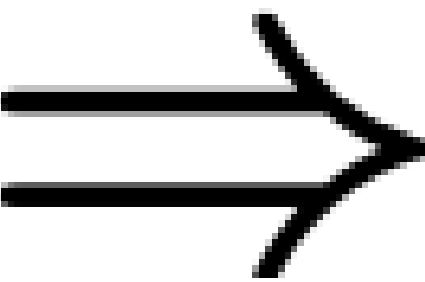
Values

Threads

0	1	4	5	8	9
2	3	6	7	10	11
12	13	16	17	20	21
14	15	18	19	22	23

((2, 2))

((2, 12))



Thr Val

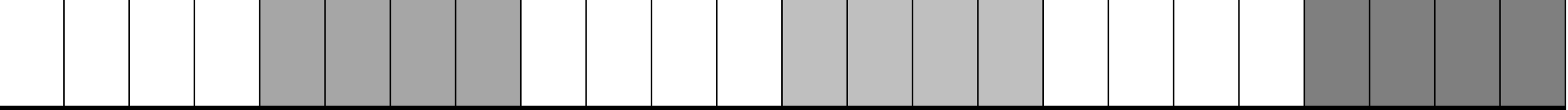
((2, 2), (2, 3))

((2, 12), (1, 4))

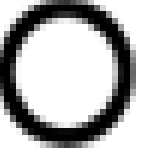
Moral of the story:

Given a layout, *partitioning* is functional *composition* followed by *slicing*.

```
Tensor tensor = make_tensor(. . .);
Tensor tv_tensor = composition(tensor, thr_val);
Tensor thr_subtensor = tv_tensor(tid, _);
```

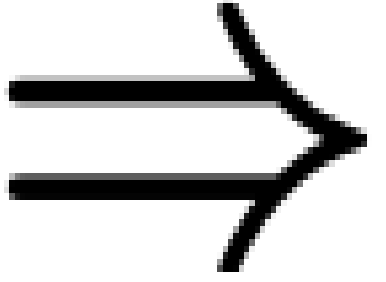


data <- coord

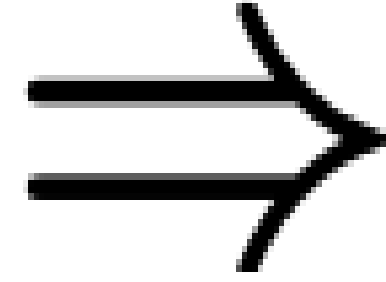


0	1	4	5	8	9
2	3	6	7	10	11
12	13	16	17	20	21
14	15	18	19	22	23

coord <- (tid, vid)



data <- (tid, vid)



data <- vid

	0	1	2	3	4	5	6	7
0	a	e	i	m	q	u	y	γ
1	b	f	j	n	r	v	z	δ
2	c	g	k	o	s	w	α	ϵ
3	d	h	l	p	t	x	β	η

4x8 data
Any layout

O

	0	1	2	3
0	0	4	16	20
1	8	12	24	28
2	1	5	17	21
3	9	13	25	29
4	2	6	18	22
5	10	14	26	30
6	3	7	19	23
7	11	15	27	31

8x4 thr-val -> 4x8 1d coord
Shape ((2,4),(2, 2))
Stride ((8,1),(4,16))



	0	1	2	3
0	a	e	q	u
1	i	m	y	γ
2	b	f	r	v
3	j	n	z	δ
4	c	g	s	w
5	k	o	α	ϵ
6	d	h	t	x
7	l	p	β	η

8x4 thr-val -> 4x8 data

Volta FP16 8x8x4 MMA

```
template <>
struct MMA_Traits<SM70_8x8x4_F32F16F16F32_NT>
{
    using ElementDVal = float;
    using ElementAVal = half_t;
    using ElementBVal = half_t;
    using ElementCVal = float;

    using Shape_MNK = Shape<_8,_8,_4>;

    using ThrID = Layout<Shape <_4,_2>,
                        Stride<_1,_16>>;

    // (T8,V4) -> (M8,K4)
    using ALayout = Layout<Shape <Shape <_4,_2>,_4>,
                        Stride<Stride<_8,_4>,_1>>;

    // (T8,V4) -> (N8,K4)
    using ALayout = Layout<Shape <Shape <_4,_2>,_4>,
                        Stride<Stride<_8,_4>,_1>>;

    // (T8,V8) -> (M8,N8)
    using CLayout = Layout<Shape <Shape <_2,_2,_2>,Shape <_2,_2,_2>>,
                        Stride<Stride<_1,_16,_4>,Stride<_8,_2,_32>>>;
};
```

	0	1	2	3	4	5	6	7
0	T0 V0	T0 V1	T0 V2	T0 V3	T16 V0	T16 V1	T16 V2	T16 V3
1	T1 V0	T1 V1	T1 V2	T1 V3	T17 V0	T17 V1	T17 V2	T17 V3
2	T2 V0	T2 V1	T2 V2	T2 V3	T18 V0	T18 V1	T18 V2	T18 V3
3	T3 V0	T3 V1	T3 V2	T3 V3	T19 V0	T19 V1	T19 V2	T19 V3

	0	1	2	3
0	T0 V0	T1 V0	T2 V0	T3 V0
1	T0 V1	T1 V1	T2 V1	T3 V1
2	T0 V2	T1 V2	T2 V2	T3 V2
3	T0 V3	T1 V3	T2 V3	T3 V3
4	T16 V0	T17 V0	T18 V0	T19 V0
5	T16 V1	T17 V1	T18 V1	T19 V1
6	T16 V2	T17 V2	T18 V2	T19 V2
7	T16 V3	T17 V3	T18 V3	T19 V3

	0	1	2	3	4	5	6	7
0	T0 V0	T0 V1	T2 V0	T2 V1	T0 V4	T0 V5	T2 V4	T2 V5
1	T1 V0	T1 V1	T3 V0	T3 V1	T1 V4	T1 V5	T3 V4	T3 V5
2	T0 V2	T0 V3	T2 V2	T2 V3	T0 V6	T0 V7	T2 V6	T2 V7
3	T1 V2	T1 V3	T3 V2	T3 V3	T1 V6	T1 V7	T3 V6	T3 V7
4	T16 V0	T16 V1	T18 V0	T18 V1	T16 V4	T16 V5	T18 V4	T18 V5
5	T17 V0	T17 V1	T19 V0	T19 V1	T17 V4	T17 V5	T19 V4	T19 V5
6	T16 V2	T16 V3	T18 V2	T18 V3	T16 V6	T16 V7	T18 V6	T18 V7
7	T17 V2	T17 V3	T19 V2	T19 V3	T17 V6	T17 V7	T19 V6	T19 V7

SM70_8x8x4_F32F16F16F32_NT

The Tiler and the TVLayout

The partitioning two-step

- For arbitrarily sized tensors $gC = (M, N)$
 - Step 1: Hit it with a Tiler to extract out 8x8 subtiles
 - Step 2: Permute 8x8 Tile with TVLayout for partitioning

T0 V0	T0 V1	T1 V0	T1 V1	T2 V0	T2 V1	T3 V0	T3 V1
T4 V0	T4 V1	T5 V0	T5 V1	T6 V0	T6 V1	T7 V0	T7 V1
T8 V0	T8 V1	T9 V0	T9 V1	T10 V0	T10 V1	T11 V0	T11 V1
T12 V0	T12 V1	T13 V0	T13 V1	T14 V0	T14 V1	T15 V0	T15 V1
T16 V0	T16 V1	T17 V0	T17 V1	T18 V0	T18 V1	T19 V0	T19 V1
T20 V0	T20 V1	T21 V0	T21 V1	T22 V0	T22 V1	T23 V0	T23 V1
T24 V0	T24 V1	T25 V0	T25 V1	T26 V0	T26 V1	T27 V0	T27 V1
T28 V0	T28 V1	T29 V0	T29 V1	T30 V0	T30 V1	T31 V0	T31 V1

Tiler = (8, 8)

TVLayout = ((4, 8), 2) : ((16, 1), 8)

```
>>> gC = make_tensor(Layout({96,192}), {192,1}), dtype=ctypes.c_float)
>>> gC
Tensor(Array(0x00000000cdafcc0, c_float), (96, 192):(192, 1))
>>>
>>> gC_tiled = zipped_divide(gC, (8,8))
>>> gC_tiled
Tensor(Array(0x00000000cdafcc0, c_float), ((8, 8), (12, 24)):({(192, 1), (1536, 8)}))
>>>
>>> composition(gC_tiled, (MyTVLayout, None))
Tensor(Array(0x00000000cdafcc0, c_float), (((4, 8), 2), (12, 24)):({(2, 192), 1), (1536, 8)}))
```


The Tiler and the TVLayout

The partitioning two-step

- For arbitrarily sized tensors $gC = (M, N)$
 - Step 1: Hit it with a Tiler to extract out 8x8 subtiles
 - Step 2: Permute 8x8 Tile with TVLayout for TV-partitioning

T0 V0	T0 V1	T1 V0	T1 V1	T2 V0	T2 V1	T3 V0	T3 V1
T4 V0	T4 V1	T5 V0	T5 V1	T6 V0	T6 V1	T7 V0	T7 V1
T8 V0	T8 V1	T9 V0	T9 V1	T10 V0	T10 V1	T11 V0	T11 V1
T12 V0	T12 V1	T13 V0	T13 V1	T14 V0	T14 V1	T15 V0	T15 V1
T16 V0	T16 V1	T17 V0	T17 V1	T18 V0	T18 V1	T19 V0	T19 V1
T20 V0	T20 V1	T21 V0	T21 V1	T22 V0	T22 V1	T23 V0	T23 V1
T24 V0	T24 V1	T25 V0	T25 V1	T26 V0	T26 V1	T27 V0	T27 V1
T28 V0	T28 V1	T29 V0	T29 V1	T30 V0	T30 V1	T31 V0	T31 V1

Tiler = (8, 8)

TVLayout = ((4, 8), 2) : ((16, 1), 8)

```
>>> gC = make_tensor(Layout({96,192}), {192,1}), dtype=ctypes.c_float)
>>> gC
Tensor(Array(0x00000000cdafcc0, c_float), (96, 192):(192, 1))
>>>
>>> gC_tiled = zipped_divide(gC, (8,8))
>>> gC_tiled
Tensor(Array(0x00000000cdafcc0, c_float), ((8, 8), (12, 24)):({192, 1}, {1536, 8}))
>>>
>>> composition(gC_tiled, (MyTVLayout, None))
Tensor(Array(0x00000000cdafcc0, c_float), (((4, 8), 2), (12, 24)):({(2, 192), 1}, {1536, 8})))
```

$$(gC \oslash T) \circ \langle (t, v), - \rangle$$

The Tiler and the TVLayout

The partitioning two-step

- For arbitrarily sized tensors $gC = (M, N)$
 - Step 1: Hit it with a Tiler to extract out 8x8 subtiles
 - Step 2: Permute 8x8 Tile with TVLayout for TV-partitioning

T0 V0	T0 V1	T1 V0	T1 V1	T2 V0	T2 V1	T3 V0	T3 V1
T4 V0	T4 V1	T5 V0	T5 V1	T6 V0	T6 V1	T7 V0	T7 V1
T8 V0	T8 V1	T9 V0	T9 V1	T10 V0	T10 V1	T11 V0	T11 V1
T12 V0	T12 V1	T13 V0	T13 V1	T14 V0	T14 V1	T15 V0	T15 V1
T16 V0	T16 V1	T17 V0	T17 V1	T18 V0	T18 V1	T19 V0	T19 V1
T20 V0	T20 V1	T21 V0	T21 V1	T22 V0	T22 V1	T23 V0	T23 V1
T24 V0	T24 V1	T25 V0	T25 V1	T26 V0	T26 V1	T27 V0	T27 V1
T28 V0	T28 V1	T29 V0	T29 V1	T30 V0	T30 V1	T31 V0	T31 V1

Tiler = (8, 8)

TVLayout = ((4, 8), 2) : ((16, 1), 8)

```
>>> gC = make_tensor(Layout({96,192}), {192,1}), dtype=ctypes.c_float)
>>> gC
Tensor(Array(0x00000000cdafcc0, c_float), (96, 192):(192, 1))
>>>
>>> gC_tiled = zipped_divide(gC, (8,8))
>>> gC_tiled
Tensor(Array(0x00000000cdafcc0, c_float), ((8, 8), (12, 24)):({(192, 1), (1536, 8)}))
>>>
>>> composition(gC_tiled, (MyTVLayout, None))
Tensor(Array(0x00000000cdafcc0, c_float), (((4, 8), 2), (12, 24)):({(2, 192), 1}, {1536, 8})))
```

$$gC \oslash (T \circ (t, v))$$



Math Blast

A lot of questions

Tiler (often a Shape)

TVLayout

$$(c \oslash \boxed{T}) \circ \langle \boxed{(t, v)}, - \rangle$$

$$c \circ (T, T^\star) \circ \langle (t, v), - \rangle$$

$$(t', v') = T \circ (t, v)$$

$$c \circ ((t', v'), (t', v')^\star)$$

$$c \oslash (t', v')$$

T0 V0	T0 V1	T1 V0	T1 V1	T2 V0	T2 V1	T3 V0	T3 V1
T4 V0	T4 V1	T5 V0	T5 V1	T6 V0	T6 V1	T7 V0	T7 V1
T8 V0	T8 V1	T9 V0	T9 V1	T10 V0	T10 V1	T11 V0	T11 V1
T12 V0	T12 V1	T13 V0	T13 V1	T14 V0	T14 V1	T15 V0	T15 V1
T16 V0	T16 V1	T17 V0	T17 V1	T18 V0	T18 V1	T19 V0	T19 V1
T20 V0	T20 V1	T21 V0	T21 V1	T22 V0	T22 V1	T23 V0	T23 V1
T24 V0	T24 V1	T25 V0	T25 V1	T26 V0	T26 V1	T27 V0	T27 V1
T28 V0	T28 V1	T29 V0	T29 V1	T30 V0	T30 V1	T31 V0	T31 V1

Math Blast

A lot of questions

Tiler (often a Shape)

TVLayout

$$(c \oslash \boxed{T}) \circ \langle \boxed{(t, v)}, - \rangle$$

... what's the complement of a Shape?

$$c \circ (T, \boxed{T^\star}) \circ \langle (t, v), - \rangle$$

... what's composition(Shape, Layout)?

$$(t', v') = T \circ (t, v)$$

... what's the complement of that??

$$c \circ ((t', v'), (t', v')^\star)$$

$$c \oslash (t', v')$$

T0 V0	T0 V1	T1 V0	T1 V1	T2 V0	T2 V1	T3 V0	T3 V1
T4 V0	T4 V1	T5 V0	T5 V1	T6 V0	T6 V1	T7 V0	T7 V1
T8 V0	T8 V1	T9 V0	T9 V1	T10 V0	T10 V1	T11 V0	T11 V1
T12 V0	T12 V1	T13 V0	T13 V1	T14 V0	T14 V1	T15 V0	T15 V1
T16 V0	T16 V1	T17 V0	T17 V1	T18 V0	T18 V1	T19 V0	T19 V1
T20 V0	T20 V1	T21 V0	T21 V1	T22 V0	T22 V1	T23 V0	T23 V1
T24 V0	T24 V1	T25 V0	T25 V1	T26 V0	T26 V1	T27 V0	T27 V1
T28 V0	T28 V1	T29 V0	T29 V1	T30 V0	T30 V1	T31 V0	T31 V1

Coordinate Strides Complete the Algebra

Better "TVLayout"s

T0 V0	T0 V1	T1 V0	T1 V1	T2 V0	T2 V1	T3 V0	T3 V1
T4 V0	T4 V1	T5 V0	T5 V1	T6 V0	T6 V1	T7 V0	T7 V1
T8 V0	T8 V1	T9 V0	T9 V1	T10 V0	T10 V1	T11 V0	T11 V1
T12 V0	T12 V1	T13 V0	T13 V1	T14 V0	T14 V1	T15 V0	T15 V1
T16 V0	T16 V1	T17 V0	T17 V1	T18 V0	T18 V1	T19 V0	T19 V1
T20 V0	T20 V1	T21 V0	T21 V1	T22 V0	T22 V1	T23 V0	T23 V1
T24 V0	T24 V1	T25 V0	T25 V1	T26 V0	T26 V1	T27 V0	T27 V1
T28 V0	T28 V1	T29 V0	T29 V1	T30 V0	T30 V1	T31 V0	T31 V1

Tiler = (8,8)
TVLayout = ((4,8),2):((16,1),8)

T0 V0	T0 V1	T1 V0	T1 V1	T2 V0	T2 V1	T3 V0	T3 V1
T4 V0	T4 V1	T5 V0	T5 V1	T6 V0	T6 V1	T7 V0	T7 V1
T8 V0	T8 V1	T9 V0	T9 V1	T10 V0	T10 V1	T11 V0	T11 V1
T12 V0	T12 V1	T13 V0	T13 V1	T14 V0	T14 V1	T15 V0	T15 V1
T16 V0	T16 V1	T17 V0	T17 V1	T18 V0	T18 V1	T19 V0	T19 V1
T20 V0	T20 V1	T21 V0	T21 V1	T22 V0	T22 V1	T23 V0	T23 V1
T24 V0	T24 V1	T25 V0	T25 V1	T26 V0	T26 V1	T27 V0	T27 V1
T28 V0	T28 V1	T29 V0	T29 V1	T30 V0	T30 V1	T31 V0	T31 V1

TVLayout = ((4,8),2):((2@1,1@0),1@1)

```
>>> gC = make_tensor(Layout((96,192), (192,1)), dtype=ctypes.c_float)
>>> gC
Tensor(Array(0x00000000cdafcc0, c_float), (96, 192):(192, 1))
>>>
>>> gC_tiled = zipped_divide(gC, (8,8))
>>> gC_tiled
Tensor(Array(0x00000000cdafcc0, c_float), ((8, 8), (12, 24)):((192, 1), (1536, 8)))
>>>
>>> composition(gC_tiled, (MyTVLayout, None))
Tensor(Array(0x00000000cdafcc0, c_float), (((4, 8), 2), (12, 24)):(((2, 192), 1), (1536, 8))))
```

```
>>> gC = make_tensor(Layout((96,192), (192,1)), dtype=ctypes.c_float)
>>> gC
Tensor(Array(0x00000000cdafcc0, c_float), (96, 192):(192, 1))
>>>
>>> TVCrLayout = composition((8,8), MyTVLayout)
>>> TVCrLayout
Layout(((4, 8), 2), ((2@1, 1@0), 1@1))
>>>
>>> zipped_divide(gC, TVCrLayout)
Tensor(Array(0x00000000cdafcc0, c_float), (((4, 8), 2), (12, 24)):(((2, 192), 1), (1536, 8))))
```

$$(gC \oslash T) \circ \langle (t, v), - \rangle$$

$$gC \oslash (T \circ (t, v))$$

Composition with Coordinate Layouts

Just selects modes

When **B**'s stride is a scaled basis element $a \cdot e_i$:

$$\mathbf{A} \circ \mathbf{B} = \mathbf{A} \circ (a \cdot e_i) \circ s = \mathbf{A}_i \circ a \circ s = \mathbf{A}_i \circ \mathbf{B}'$$

— Whitepaper §3.3.2

...which collapses to the rank-1, integral base case.

```
>>> A = Layout((4, 5), (5, 1))
>>> composition(A, Layout(4, E(0)))           # pick mode 0
Layout(4, 5)
>>> composition(A, Layout(5, E(1)))           # pick mode 1
Layout(5, 1)
>>> composition(A, Layout(2, 2*E(0)))         # pick mode 0, stride 2
Layout(2, 10)
```


What are Shapes?

Shapes were coordinate layouts the whole time

$$(4, 8) \equiv \langle 4, 8 \rangle \equiv (4:1, 8:1) \equiv (4, 8):(e_0, e_1)$$

- The action of a shape (idx2crd) is equivalent to the action of a coordinate layout.

```
>>> tiler_to_layout((4, 5)) # (4,5)
Layout((4, 5), (1@0, 1@1))
>>> tiler_to_layout((Layout(4, 1), Layout(5, 1))) # (4:1,5:1)
Layout((4, 5), (1@0, 1@1))
>>> tiler_to_layout(Layout((4,5), (E(0), E(1)))) # (4,5):(1@0,1@1)
Layout((4, 5), (1@0, 1@1))

>>> zipped_divide(Layout((40,40),(42,1)), (4, 5))
Layout(((4, 5), (10, 8)), ((42, 1), (168, 5)))
>>> zipped_divide(Layout((40,40),(42,1)), (Layout(4, 1), Layout(5, 1)))
Layout(((4, 5), (10, 8)), ((42, 1), (168, 5)))
>>> zipped_divide(Layout((40,40),(42,1)), Layout((4,5), (E(0), E(1))))
Layout(((4, 5), (10, 8)), ((42, 1), (168, 5)))
```

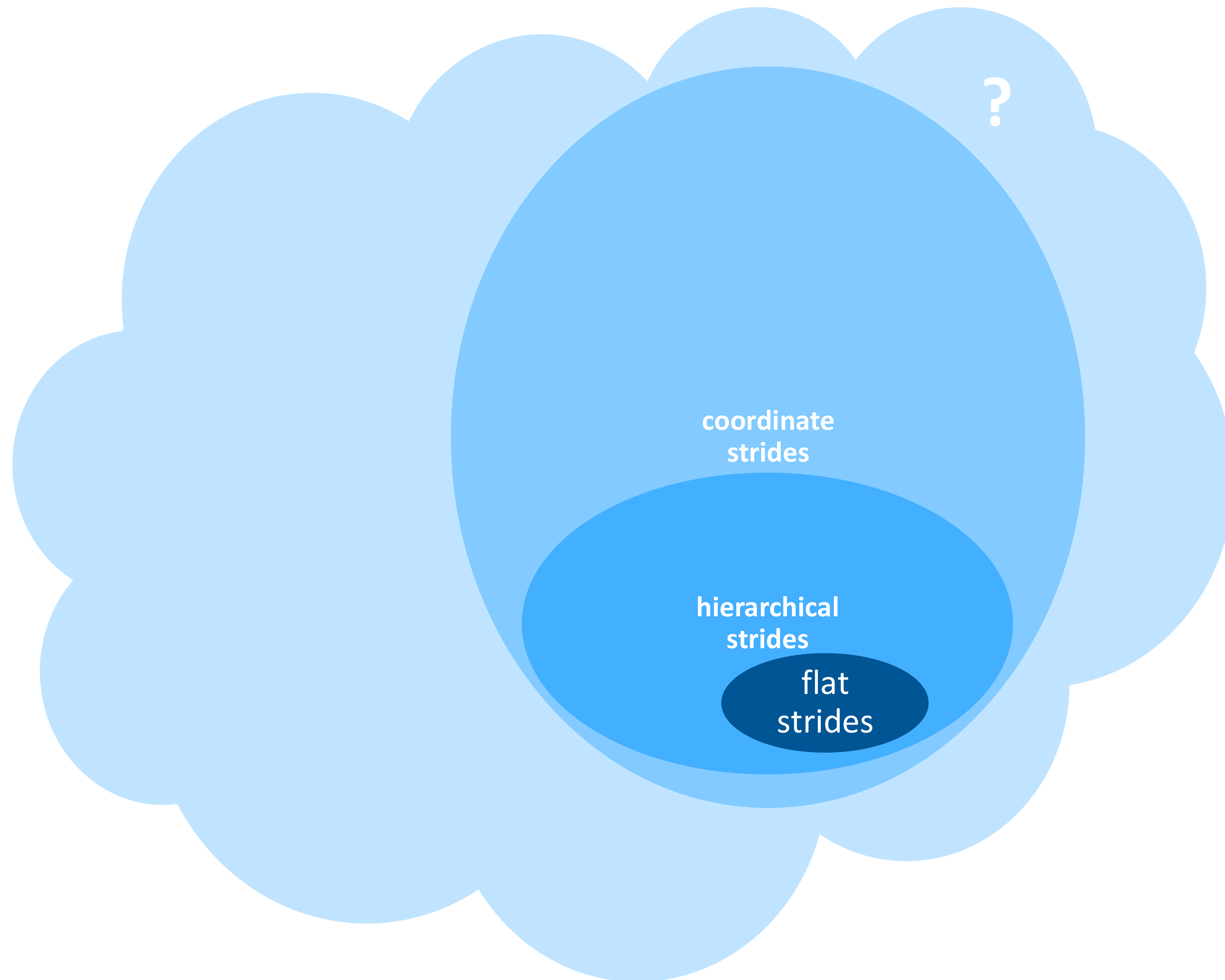

Coordinate Strides Complete the Algebra

Full Support

Operation	$\mathbb{Z}$	$\mathbb{Z}^S$
<code>coalesce</code>	·	·
<code>RHS composition(A, B)</code>	·	·
<code>LHS composition(A, B)</code>	·	·
<code>complement</code>	·	·
<code>logical divide / product</code>	·	·
<code>right inverse</code>	·	·
<code>left inverse</code>	·	·
<code>coshape</code>	·	·

The representable set?

Coordinate strides...



Bring Your Own stride

Just + and *

```
class Zmod(StrideScalar):
    """Integers mod n: the codomain of a ring buffer, or a circular convolution."""
    def __init__(self, v, n):
        self.v, self.n = v % n, n

    def __add__(self, other):
        if other == 0: return self # int 0 is the identity
        if isinstance(other, Zmod) and self.n == other.n:
            return Zmod(self.v + other.v, self.n)
        return NotImplemented
    __radd__ = __add__

    def __mul__(self, other):
        if not isinstance(other, int): return NotImplemented
        return Zmod(self.v * other, self.n)
    __rmul__ = __mul__
```


And (a lot) just runs

Abstracted strides

	0	1	2	3	4	5
0	0	3	6	1	4	7
1	1	4	7	2	5	0
2	2	5	0	3	6	1
3	3	6	1	4	7	2

$(4, 6) : (\text{Zmod}(1, 8), \text{Zmod}(3, 8))$

- Offsets that **wrap**
- Symbolic strides come free as well – `sympy.Expr`
 - Placeholders for unknown/dynamic values

```
L = Layout((4, 8), (1, m))  
L(2, 3) => 3*m + 2
```




All about Swizzles

History of Swizzles

PISL, PDSL and Now

- The first swizzle was the ComposedLayout form,

$$\text{Swizzle}\langle B, M, S \rangle \circ \text{Offset} \circ \text{Layout}$$

- The Swizzle "mixes" the codomain offset relatively generically
- The Layout still allows for tiling, permutation, slicing, etc so the whole expression can still be transformed algebraically
- Simplifications to "normal layout" are possible in some situations.
- But inverses, RHS composition, etc fail without special treatment.
- Tensors then bind this to an Accessor:

$$\text{Pointer} \circ (\text{Swizzle}\langle B, M, S \rangle \circ \text{Offset} \circ \text{Layout})$$

- This became the "position-independent swizzle" layout/tensor (PISL)

History of Swizzles

PISL, PDSL and Now

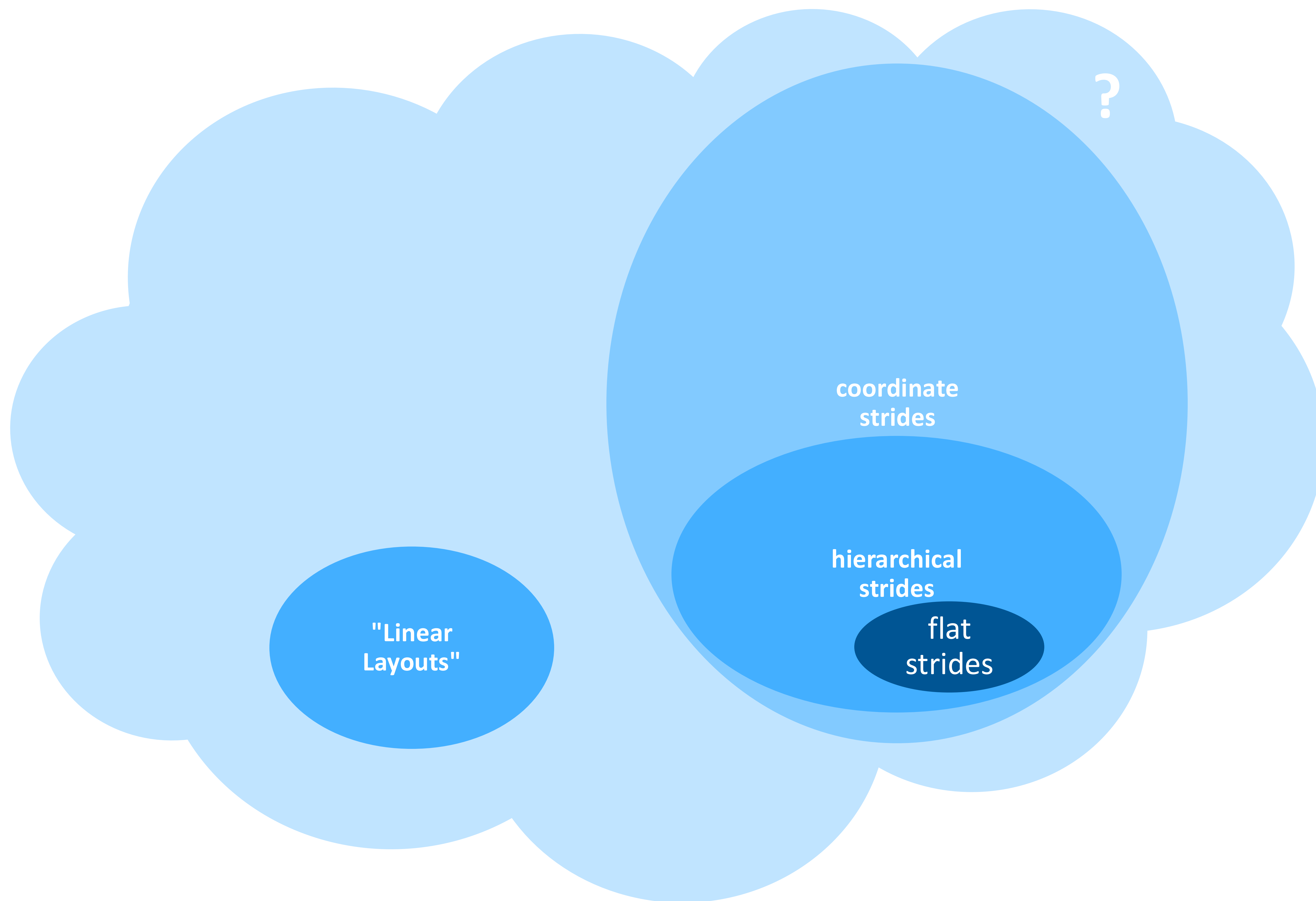
- Then, Hopper WGMMMA came along.
 - The TMA/MMA themselves "understood" swizzles... but how exactly?
- The revelation was that the hardware was swizzling the **address itself**
 - So the Tensor model then uses a swizzled-pointer:

$$(\text{Swizzle}\langle B, M, S \rangle \circ \text{Pointer}) \circ \text{Layout}$$

- This became the "position-dependent swizzle" layout/tensor (PDSL)
- It has a normal layout with full algebraic support, but modified accessor
 - Descriptor-based instructions love this
 - Thread-independent accesses don't (swizzle is evaluated each dereference, no simplifications when pointer is unknown)
- PISL $\leftrightarrow$ PDSL only when the pointer/offset is sufficiently aligned.
 - When you can prove the swizzle permutes with the pointer:

$$\text{Swizzle}\langle B, M, S \rangle \circ \text{Pointer} == \text{Pointer} \circ \text{Swizzle}\langle B, M', S \rangle$$

The representable set?



F2 Strides

Mixing within a Layout evaluation

The field $F_2 = (0, 1, \text{XOR}, \text{AND})$ of arithmetic operations modulo 2 is an integer-semimodule.

— Whitepaper §2.3.1

```
>>> F2(1) + F2(8)          # XOR
F9
>>> F2(3) + F2(3)          # every element is its own inverse
F0
>>> 3 * F2(3)              # carry-less: 3 ^ 6 = 5, not 9
F5
>>> divmod(F2(0b1011), 0b11) # polynomial long division
(F6, F1)
```


F2 and SMEM Bank Conflicts

Swizzles

	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	8	9	10	11	12	13	14	15
2	16	17	18	19	20	21	22	23
3	24	25	26	27	28	29	30	31
4	32	33	34	35	36	37	38	39
5	40	41	42	43	44	45	46	47
6	48	49	50	51	52	53	54	55
7	56	57	58	59	60	61	62	63

$(8, 8) : (8, 1)$

column → **1 bank**

	0	1	2	3	4	5	6	7
0	F0	F9	F18	F27	F36	F45	F54	F63
1	F1	F8	F19	F26	F37	F44	F55	F62
2	F2	F11	F16	F25	F38	F47	F52	F61
3	F3	F10	F17	F24	F39	F46	F53	F60
4	F4	F13	F22	F31	F32	F41	F50	F59
5	F5	F12	F23	F30	F33	F40	F51	F58
6	F6	F15	F20	F29	F34	F43	F48	F57
7	F7	F14	F21	F28	F35	F42	F49	F56

$(8, 8) : (F2(1), F2(9))$

row & col → **8 banks**

	0	1	2	3	4	5	6	7
0	F0	F1	F2	F3	F4	F5	F6	F7
1	F9	F8	F11	F10	F13	F12	F15	F14
2	F18	F19	F16	F17	F22	F23	F20	F21
3	F27	F26	F25	F24	F31	F30	F29	F28
4	F36	F37	F38	F39	F32	F33	F34	F35
5	F45	F44	F47	F46	F41	F40	F43	F42
6	F54	F55	F52	F53	F50	F51	F48	F49
7	F63	F62	F61	F60	F59	F58	F57	F56

$(8, 8) : (F2(9), F2(1))$

row & col → **8 banks**

- No Swizzle<B,M,S>. No ComposedLayout. No special casing.
- A swizzle is a layout whose strides happen to be F2.

Semilinearity

Everything is a Matrix-product

In natural coordinates, evaluation is linear:

$$\mathbf{L}(c) = (D \circ S)(c) = d \cdot \tilde{c} \quad \mathbf{L}(\alpha \tilde{c}_0 + \beta \tilde{c}_1) = \alpha \mathbf{L}(\tilde{c}_0) + \beta \mathbf{L}(\tilde{c}_1)$$

so $d \cdot \tilde{c}$ is a **generalized matrix-vector product**

$\mathbf{L}$	Linear Form: $r = \mathbf{D} \tilde{c}$	Comment
$((2, 2), (4, 2)) : ((1, 8), (2, 16))$	$r = [1 \ 8 \ 2 \ 16] \begin{bmatrix} \tilde{c}_0 \\ \tilde{c}_1 \\ \tilde{c}_2 \\ \tilde{c}_3 \end{bmatrix}$	Integer strides are columns of $1 \times n \mathbb{Z}$ -matrix
$(4, (4, 2)) : (e_1, (e_0, 6e_1))$	$\begin{bmatrix} r_0 \\ r_1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 6 \end{bmatrix} \begin{bmatrix} \tilde{c}_0 \\ \tilde{c}_1 \\ \tilde{c}_2 \end{bmatrix}$	Coordinate strides are columns of $m \times n \mathbb{Z}$ -matrix
$(4, 4) : (f_1, f_5)$ $\equiv ((2, 2), (2, 2)) : ((f_1, f_2), (f_5, f_{10}))$	$\begin{bmatrix} r_0 \\ r_1 \\ r_2 \\ r_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tilde{c}_0 \\ \tilde{c}_1 \\ \tilde{c}_2 \\ \tilde{c}_3 \end{bmatrix}$	Binary strides are columns of $m \times n \mathbb{F}_2$ -matrix

Table 1: Layouts and their associated linear forms.

The F2 Matrix

Everything is a Matrix-product

$$(4, 4):(f_1, f_5) \equiv ((2, 2), (2, 2)):((f_1, f_2), (f_5, f_{10}))$$

$$\begin{bmatrix} r_0 \\ r_1 \\ r_2 \\ r_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tilde{c}_0 \\ \tilde{c}_1 \\ \tilde{c}_2 \\ \tilde{c}_3 \end{bmatrix}$$

```
>>> L1 = Layout((4, 4), (F2(1), F2(5)))
>>> L2 = Layout(((2, 2), (2, 2)), ((F2(1), F2(2)), (F2(5), F2(10))))
>>> [int(L1(i)) for i in range(16)] == [int(L2(i)) for i in range(16)]
True
>>> coalesce(L2) == L1           # not just the same map -- the same layout
True
>>> [int(L1(i)) for i in range(16)]
[0, 1, 2, 3, 5, 4, 7, 6, 10, 11, 8, 9, 15, 14, 13, 12]
```

- Read off the columns from the refined layouts: f_1, f_2, f_5, f_{10}
- `coalesce` just works

F2 Layout Inverses

Just matrix inverse

`right_inverse(A)` is the largest injective `R` with `A(R(i)) == i`, built by following each codomain axis up its chain of strides:

Over `F2` a stride may additionally carry any component the covered modes already span, since XOR-ing those bits permutes them rather than colliding, so swizzles invert too. — `algebra.py`, `right_inverse`

Layout	<code>right_inverse</code>	Reading
<code>(8, 8) : (F1, F9)</code>	<code>(8, 8) : (F1, F9)</code>	its own inverse
<code>(4, 4) : (F1, F5)</code>	<code>(4, 4) : (F1, F5)</code>	and so is the 4×4 swizzle
<code>(8, 8) : (F9, F1)</code>	<code>(8, 8) : (F8, F9)</code>	transposed: still bijective, no longer self-inverse
<code>(4, (4, 3)) : (F1, (F5, F16))</code>	<code>(4, 4, 3) : (F1, F5, F16)</code>	Fig 6: bijective, hierarchy flattened
<code>(8, 8) : (F1, F13)</code>	<code>8 : F1</code>	<code>4 * F13</code> escapes the field <code>F8</code> opens — chain stops
<code>(4, 4) : (F1, F1)</code>	<code>4 : F1</code>	not injective: 4 distinct images of 16

- Note that this uses

$$\text{idx2crd}(\text{F2}(0\text{b}10110), (4, 8)) \Rightarrow (\text{F2}, \text{F5})$$

This is BCP / BMMC

Existing literature

The F_2 linear forms are exactly the classical index transforms:
BCP, Bit Permute Complement, and **BMMC**, Bit Matrix Multiply Complement.

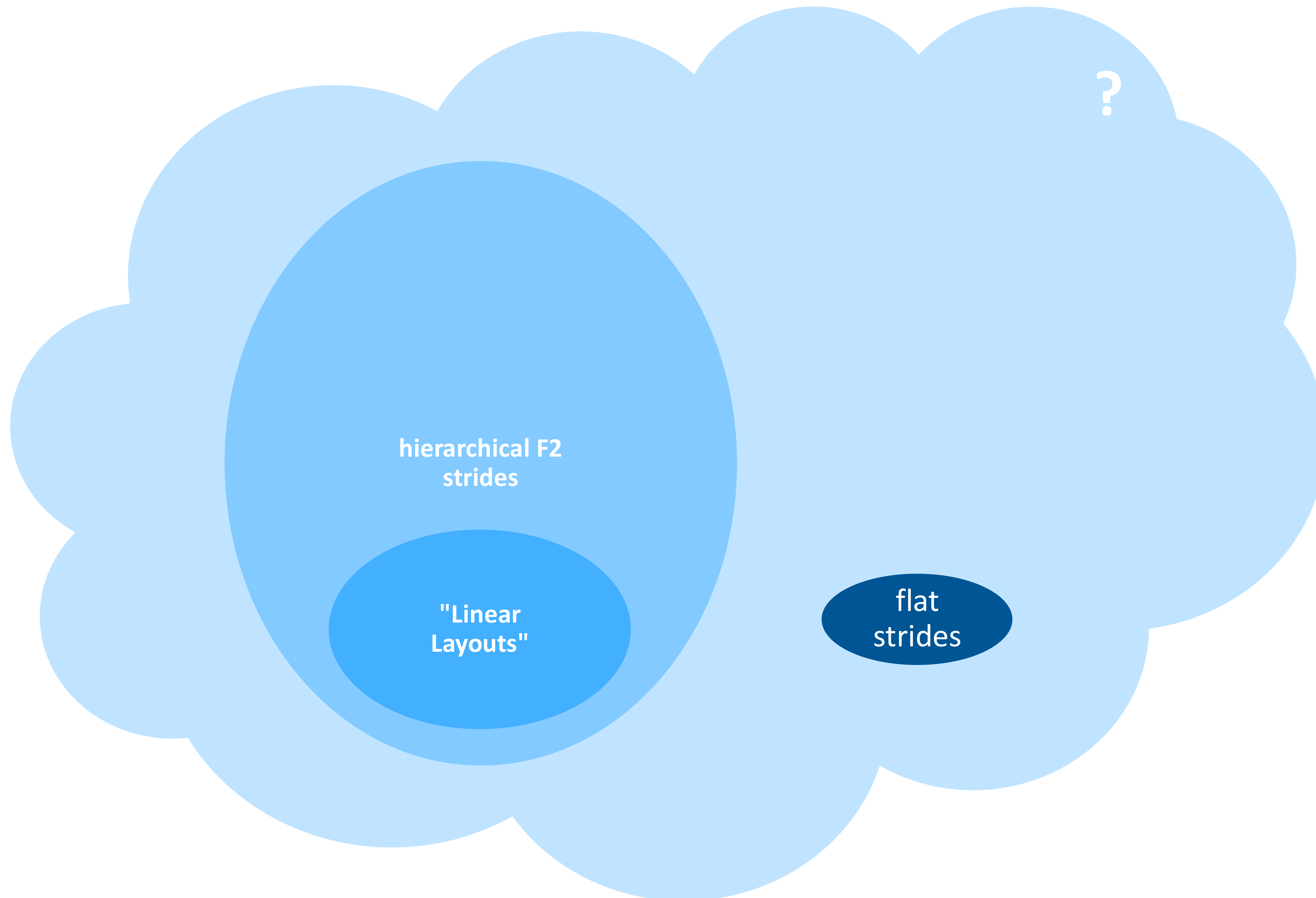
Edelman et al.; Cormen; and recently Tillet's *Linear Layouts* for SIMT GPUs

The correspondence runs deeper than the representation:

Layout algebra	Linear algebra
composition	matrix multiplication
right_inverse / left_inverse	Moore–Penrose pseudo-inverses
complement	completing a basis
logical_divide / logical_product	factorization

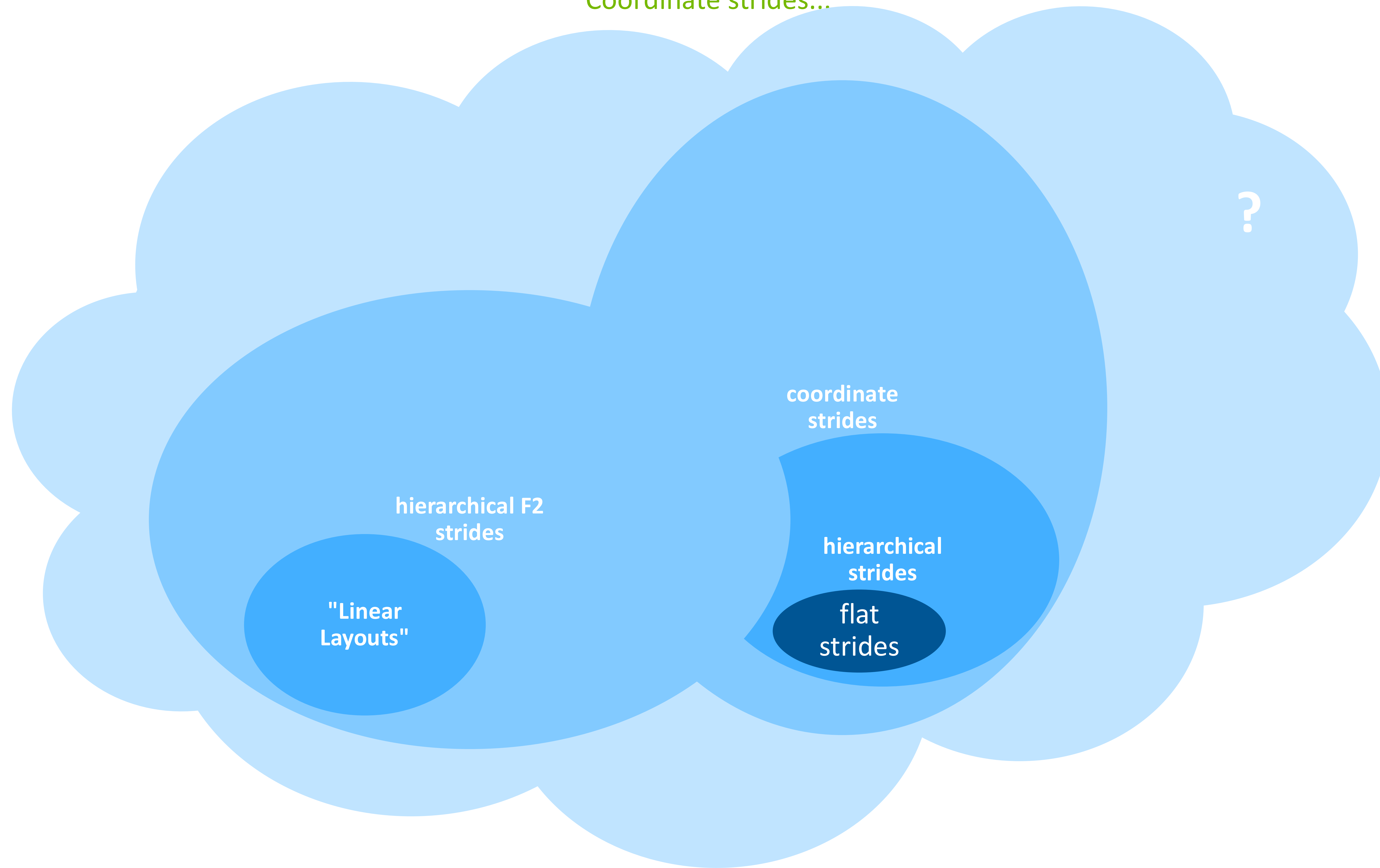
The representable set?

Power-of-2 F2 Layouts are isomorphic to Linear Layouts



The representable set?

Coordinate strides...



Layouts are Complete

But sacrifice some of the algebra

2.4.3 Completeness

Every function f with $f(0) = 0$ and finite domain $\mathbb{Z}_N$ can be represented as the functional composition of the finite sequence of CUTE layouts. This means that CUTE layouts are a generating set under functional composition. Such a function f can be represented as sequence of compositions

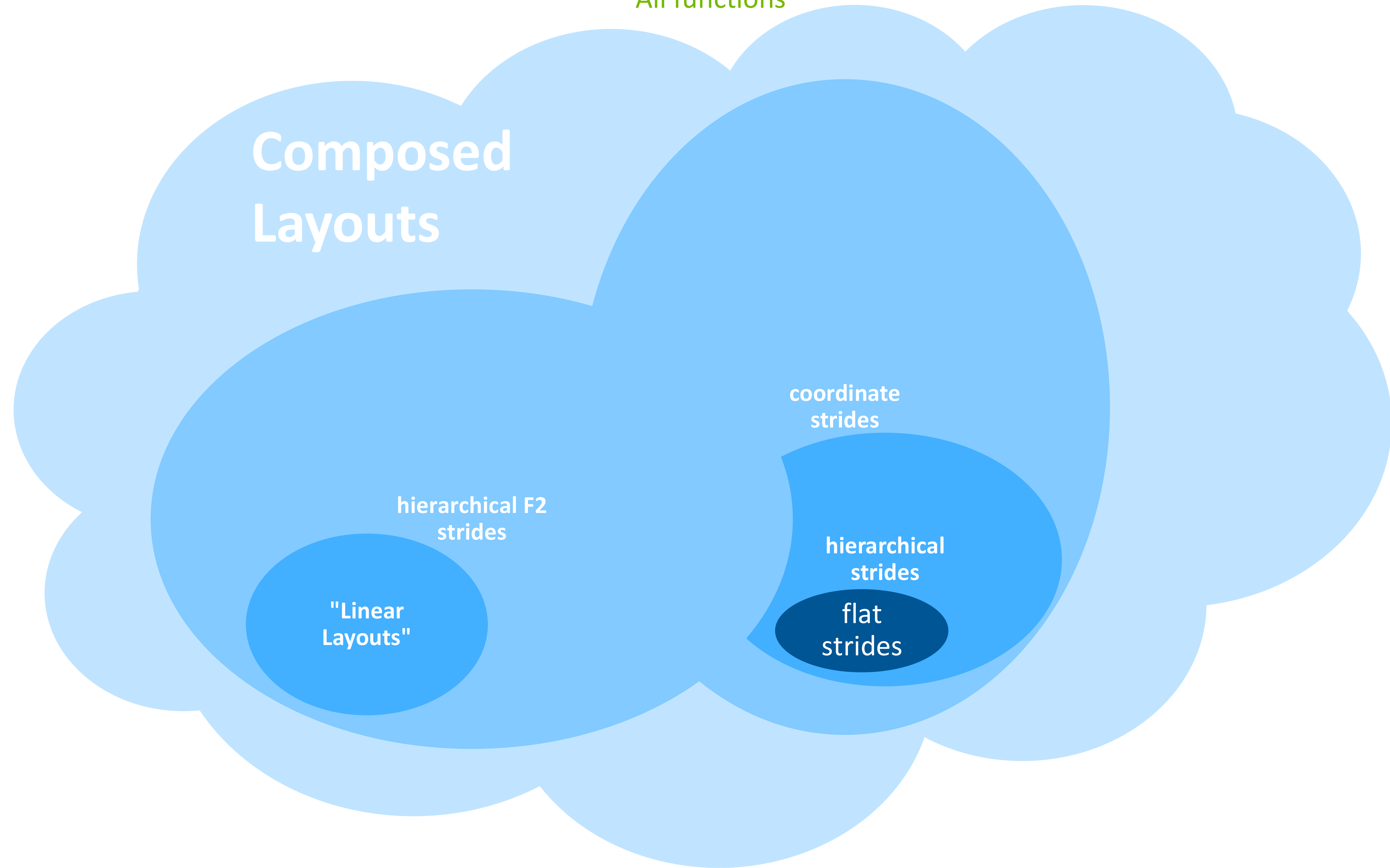
$$f \equiv (2, 2, 2, \dots) : (f(1), f(2), f(3), \dots) \circ (3, 1) : (1, 4) \circ (4, 1) : (1, 6) \circ \dots \circ (N - 1, 1) : (1, 2(N - 2)).$$

For all $i \in \mathbb{Z}_N / \{0\}$, the rightmost $N - 3$ layouts map $i \rightarrow 2^{i-1}$ and the leftmost layout maps $2^{i-1} \rightarrow f(i)$. Note that we've used the extended domain, $\mathbb{Z}$, rather than the in-bounds domain, $\mathbb{Z}_N$, to evaluate the intermediate layouts above.

- These are the so-called ComposedLayouts
- In general,
 - No complement (LHS product, RHS divide)
 - No inverses
- But retain
 - Tiling (LHS divide)
 - Permutation (composition)
 - Slicing

The representable set?

All functions



Admissibility

Not everything is in yet

Operation	$\mathbb{Z}$	$\mathbb{Z}^S$	$\mathbb{F}_2^m$	What it needs
coalesce	✓	✓	✓	leaf <code>+</code> , <code>*</code> only
make_layout (<i>concatenate</i>)	✓	✓	✓	<i>one</i> semimodule for all sublayouts
coshape	✓	✓	✓	monotone <code>+</code> , or a <code>_coshape_bound</code>
right_inverse	✓	✓	✓	injectivity (XOR gets <i>extra</i> room)
logical_divide	✓	✓	✓	complements the tiler , not <code>A</code>
composition(<code>A</code> , <code>B</code>) — LHS <code>A</code>	✓	✓	✓	<code>A</code> 's strides are only scaled and added
composition(<code>A</code> , <code>B</code>) — RHS <code>B</code>	✓	✓	✗	<code>B</code> 's stride must divide <code>A</code> 's shape
complement	✓	✓	✗	a total order on the image
logical_product	✓	✓	✗	complements <code>A</code>
left_inverse	✓	✓	✗	stride quotients must be integers



Ongoing Research

Mixed-stride Layouts

Already function

```
>>> L = Layout((8, 8), (F2(3) * E(0), 7 * E(1)))
>>> L
Layout((8, 8), (F3@0, 7@1))
>>> L(2, 3)
(F6, 21)                # axis 0 carry-less 2*3, axis 1 an ordinary 3*7
```

- The ArithTuple (coordinate strides) can contain any scalar and defer to that algebra.
- Two different types of strides can't add, but they can both participate in a product type like this.
- Various algebraic operations fail
 - LHS composition (tiling, permutation, slicing) is always available.

Coming Soon

IOUs

- Continued algebraic, F2, and swizzle development
- GEMM alg tutorial
 - With CONV and im2col/stencil_divide implementation
- SMEM Solver
 - Given two TVLayouts (of registers, for example), what is the best SMEM Layout to write to and read from?
- Integrations
 - CuTeDSL+PyCuTe examples
 - Cuda.lang metaprogramming
- C++ reference implementation?

